

ДИПЛОМНА РАБОТА

Тема: Система за автоматизирана търговия, наблюдение и анализ на крипто активи

Дипломант: Йоана Калинова Йотова

Фак. №: 471222002

Специалност: Информатика и Софтуерни Науки(ИСН)

Образователно- квалификационна степен: **бакалавър**

Дипломен ръководител: доц. д-р Златко Захариев - ФПМИ

София 2026

СЪДЪРЖАНИЕ

ГЛАВА 1. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ИЗИСКВАНИЯТА КЪМ СИСТЕМАТА

1.1. Автоматизирана търговия с крипто активи.....	5
1.2. Основни понятия.....	5
1.3. Жизнен цикъл на търговския сигнал.....	5
1.4. Функционални изисквания.....	6
1.5. Нефункционални изисквания.....	6
1.6. Изисквания към безопасността на търговския процес.....	7
1.7. Среда за изпълнение и контролирано въвеждане.....	7
1.8. Изисквания към операторския интерфейс и обобщение.....	7

ГЛАВА 2. АРХИТЕКТУРА НА СИСТЕМАТА

2.1. Архитектурен подход.....	8
2.2. Основни компоненти и отговорности.....	8
2.3. Runtime поток от пазарно събитие до изпълнение.....	10
2.4. Комуникация чрез Redis.....	10
2.5. PostgreSQL и трайното състояние.....	11
2.6. Strategy, Risk и Execution като отделни архитектурни граници.....	11
2.7. Външни и вътрешни източници на сигнали.....	11
2.8. Paper, Demo и Production-ready execution.....	11
2.9. Reconciliation като архитектурен защитен слой.....	12
2.10. Research и аналитичен слой.....	12
2.11. Control plane и операторски интерфейс.....	12
2.12. Observability и operational слой.....	12
2.13. Обобщение на архитектурните решения.....	12

ГЛАВА 3. ТЕХНОЛОГИЧНА ОСНОВА И ИНЖЕНЕРНИ РЕШЕНИЯ

3.1. Критерии при избора на технологичния стек.....	14
3.2. C# и .NET 9 като backend основа.....	14
3.3. BackgroundService и lifecycle на runtime услугите.....	14
3.4. ASP.NET Core и application boundary.....	14
3.5. Authentication, authorization и сигурност на операторския интерфейс.....	15
3.6. Redis като communication и coordination слой.....	15
3.7. PostgreSQL, Npgsql и Dapper.....	15
3.8. SQL migrations и versioning на схемата.....	15
3.9. Binance Futures REST API.....	16
3.10. WebSocket потоци: MarketData и UserStream.....	16
3.11. React, TypeScript и TanStack Query.....	16
3.12. Docker и Docker Compose.....	16
3.13. Контейнеризирано разгръщане върху Synology NAS.....	16
3.14. Наблюдаемост: health, heartbeat, metrics и notifications.....	17
3.15. xUnit тестове.....	17
3.16. Основни технологични компромиси.....	17

ГЛАВА 4. РЕАЛИЗАЦИЯ НА ОСНОВНИТЕ МЕХАНИЗМИ НА СИСТЕМАТА

4.1. Организация на реализацията.....	18
4.2. TradingView като външен източник на сигнали.....	18
4.3. Вътрешно генериране на сигнали.....	18
4.4. Runtime конфигурация и кеширане.....	19
4.5. TradingEngine и последователност на защитните проверки.....	19
4.6. Идемпотентност, operation lock, cooldown и signal throttle.....	20

4.7. BOT8012, BOT8013 и BOT8014 – TP-only Grid.....	20
4.8. BOT8011 и BOT8015 – ProtectedStop3.....	21
4.9. BOT8016 – Alligator Multi-Timeframe.....	21
4.10. Paper Trading като реален execution режим.....	21
4.11. Binance execution и uncertain outcome.....	22
4.12. UserStream и последователността на lifecycle събитията.....	22
4.13. Retry, permanent failure и странични ефекти.....	23
4.14. Reconciliation и healing.....	23
4.15. Telegram и operational notifications.....	24
4.16. Event Store и проследимост.....	24
4.17. Обобщение на failure поведението.....	24

ГЛАВА 5. PERSISTENCE, АНАЛИЗ, ОПТИМИЗАЦИЯ И ОПЕРАТОРСКИ КОНТРОЛ

5.1. Поля на persistence слой.....	26
5.2. Trading history и проследимост на една търговска операция.....	26
5.3. Event Store, projections и причинно-следствена връзка.....	26
5.4. Paper Trading state и изолация на тестовата среда.....	27
5.5. Risk, reconciliation и operational данни.....	27
5.6. Performance analytics и IPerformanceAnalyticsStore.....	27
5.7. Optimization процес и parameter search.....	28
5.8. Walk-forward анализ и ограничаване на overfitting.....	28
5.9. Backtesting и optimization като versioned run-ове.....	28
5.10. Jobs Worker и трайният lifecycle на дългите операции.....	29
5.11. Dashboard Application слой и query projections.....	29
5.12. Dashboard Web като визуализация на projections.....	30
5.13. Runtime configuration, versioning и audit reason.....	30
5.14. Операторски команди и command history.....	30
5.15. Authentication, authorization и API hardening.....	31
5.16. Observability данни и service health.....	31
5.17. Database migrations и надеждност на deployment-a.....	31
5.18. Deployment върху Docker Compose и Synology NAS.....	31

ГЛАВА 6. ТЕСТВАНЕ И ПРАКТИЧЕСКА ВЕРИФИКАЦИЯ

6.1. Подход към практическата верификация.....	33
6.2. Проверка на стартирането и инфраструктурната готовност.....	33
6.3. Проверка на runtime управлението и конфигурацията на BOT8012.....	34
6.4. Проверка на TradingView сигнал и TradingEngine pipeline.....	35
6.5. Binance Demo end-to-end сценарий с BOT8012.....	35
6.6. UserStream и последващ lifecycle на борсовата операция.....	37
6.7. Persistence, Dashboard projections и audit trail.....	37
6.8. Проверка на Paper Trading режима.....	38
6.9. Проверка на историческите пазарни данни.....	39
6.10. Проверка на operational alerts и failure visibility.....	39
6.11. Backtesting на BOT8012.....	40
6.12. Optimization на стратегическите параметри.....	40
6.13. Replay и детерминистично възпроизвеждане.....	41
6.14. Проверка на идемпотентност, cooldown и risk ограничения.....	43
6.15. Reconciliation и възстановяване след промяна на runtime състоянието.....	43
6.16. Обобщение на проверените практически сценарии.....	43

ГЛАВА 7. ОЦЕНКА НА СИСТЕМАТА, ОГРАНИЧЕНИЯ И НАСОКИ ЗА РАЗВИТИЕ

7.1. Оценка на постигнатата реализация.....	47
7.2. Силни страни на архитектурата.....	47

7.3. Ограничения на използвания комуникационен модел.....	47
7.4. Ограничения на историческия анализ.....	47
7.5. Fail-closed като принцип на безопасност.....	48
7.6. Операторски интерфейс и човешки фактор.....	48
7.7. Посоки за развитие на event delivery и persistence.....	48
7.8. Посоки за развитие на reconciliation и recovery.....	48
7.9. Развитие на risk management.....	49
7.10. Развитие на analytics и optimization.....	49
7.11. Production-ready deployment.....	49
ЗАКЛЮЧЕНИЕ.....	50
ИЗПОЛЗВАНА ЛИТЕРАТУРА И ИЗТОЧНИЦИ.....	52
ПРИЛОЖЕНИЕ.....	53

ГЛАВА 1. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ИЗИСКВАНИЯТА КЪМ СИСТЕМАТА

1.1. Автоматизирана търговия с крипто активи

Автоматизираната търговия представлява използване на софтуерни правила за наблюдение на пазара, оценка на предварително зададени условия и изпълнение на търговски действия без необходимост всяко решение да бъде въвеждано ръчно. При най-простите системи това може да бъде реакция при достигане на определена цена. При реална платформа обаче процесът включва прием на пазарни данни, изчисляване на индикатори, генериране или получаване на сигнал, проверка на текущото състояние, оценка на риска, изпълнение на поръчки и последващо проследяване на резултата.

Крипто пазарите са подходящ пример за подобна система, защото търговията е непрекъсната, цените се променят бързо, а външната борса е независим източник на данни и събития. Приложението не контролира кога ще пристигне дадено съобщение, дали мрежовата връзка ще бъде прекъсната или дали външна операция е завършила преди локалният процес да получи отговор. Затова коректното поведение не се изчерпва с логиката LONG или SHORT, а включва защита при повторени сигнали, конкурентни операции, временни грешки и несъответствие между локалното състояние и борсата.

Разработваната система е проектирана като обща платформа, в която могат да работят различни стратегии. Това поставя акцент върху повторно използваемите механизми за обработка на сигнали, риск, изпълнение, история, възстановяване и наблюдаемост, а не върху една конкретна търговска формула.

1.2. Основни понятия

Пазарната свещ (candlestick) е агрегирано представяне на движението на цената за определен интервал и обикновено съдържа начална, максимална, минимална и крайна цена, както и обем. За автоматизираната обработка е важно да се разграничава текуща от затворена свещ. Стойностите на незавършената свещ могат да се променят до края на интервала, поради което използването ѝ за окончателно решение може да доведе до нестабилни сигнали.

Техническият индикатор е изчислена стойност върху исторически пазарни данни. В системата се използват индикатори като Alligator, SMA и Bollinger Bands. Те не изпълняват сделка самостоятелно, а предоставят вход за стратегията. При multi-timeframe обработка е съществено индикаторната стойност да бъде съпоставена с правилния момент във времето, за да не се използва информация, която още не е била известна.

Търговският сигнал е формализирано намерение за действие. Той съдържа идентификатор, бот, символ, посока, време и източник. Сигналът не е равен на поръчка: след получаването му системата все още трябва да провери валидност, идемпотентност, runtime състояние, ограничения на стратегията и централизирания риск.

Поръчката е инструкция към борсата, а позицията е текущата пазарна експозиция, създадена вследствие на изпълнение. Една позиция може да бъде свързана с няколко поръчки - например entry, take-profit и stop-loss. Поради това състоянието на позицията и състоянието на поръчките трябва да се разглеждат отделно. PnL показва финансовия резултат, а при оценка на риска са важни още notional exposure, използваният маржин, ливъриджът и общата експозиция на портфейла.

1.3. Жизнен цикъл на търговския сигнал

Общият процес започва с пазарни данни. Върху тях се изчисляват индикатори, след което стратегията може да създаде вътрешен сигнал или системата да получи такъв от външен източник. Полученият TradeSignal се подава към общия търговски pipeline, където се извършват проверки за време, повторна обработка, конкурентен достъп, runtime статус и cooldown.

Ако сигналът е допустим, bot-specific policy оценява текущите позиции и решава дали конкретната стратегия разрешава нов entry. След това централният risk manager проверява общите ограничения върху броя позиции, експозицията и останалите рискови показатели. Само при положително решение сигналът достига execution слоя.

След изпълнението lifecycle-ът продължава. Борсата може да изпрати последващо събитие за fill, частично изпълнение, cancel или activation на protective order. Тези събития трябва да обновят локалното състояние. Ако то се различава от authoritative състоянието на борсата, reconciliation и healing механизмите трябва да открият и при възможност да коригират несъответствието. Така една сделка се разглежда като последователност от събития, а не като единична HTTP заявка.

1.4. Функционални изисквания

Основните функционални изисквания следват end-to-end процеса на платформата. Системата трябва да получава пазарни данни от Binance Futures[1], да поддържа различни символи и времеви интервали и да изчислява технически индикатори върху тях. Трябва да бъде възможно както генериране на вътрешни сигнали, така и приемане на външни сигнали, като двата източника използват общ TradeSignal contract.

Trading pipeline трябва да проверява валидността на сигнала, идемпотентността, cooldown, runtime конфигурацията и bot-specific правилата. След strategy approval се изисква централизирана оценка на риска и едва след това изпълнение в избраната среда. Системата трябва да поддържа Paper execution за безопасна симулация и Binance Demo/production-ready execution чрез signed API операции.

След изпълнение трябва да се обработват UserStream събития за поръчки и позиции, да се поддържа локален lifecycle и да има възможност за reconciliation след restart или открито несъответствие. Необходимо е трайно съхраняване на търговска история, lifecycle events, runtime и operational информация.

Отделна група функционалности е свързана с анализ. Historical data трябва да може да се използва за backtesting, optimization и replay. Резултатите трябва да бъдат достъпни през операторски интерфейс заедно със сигнали, позиции, health, alerts, audit и background jobs.

1.5. Нефункционални изисквания

Надеждността е основно нефункционално изискване. Временна недостъпност на Redis, PostgreSQL или Binance не трябва автоматично да води до неконтролирано повторение на външна финансова операция. Грешките трябва да бъдат разграничавани според това дали са временни, постоянни или с неизвестен резултат.

Идемпотентността е необходима както при търговските сигнали, така и при операторски команди и background jobs. Повторно получен логически request не трябва да създава втори side effect. За конкурентните случаи е необходимо отделно синхронизиране, защото два различни сигнала могат да пристигнат почти едновременно и да работят върху едно и също състояние.

Съгласуваността на състоянието също е критична. Локалната база, Redis runtime state и Binance не споделят обща транзакция, поради което временно разминаване е възможно. Архитектурата трябва да позволява откриване, регистриране и контролирано възстановяване на подобни случаи.

Системата трябва да бъде наблюдаема. Всяка основна услуга трябва да предоставя health или heartbeat информация, а значимите проблеми и операторски действия да могат да бъдат проследени чрез alerts, audit, metrics и correlation identifiers. Разширяемостта е друго важно изискване: нов bot, indicator или Dashboard feature не трябва да изисква промяна на целия runtime pipeline.

1.6. Изисквания към безопасността на търговския процес

Тъй като системата може да изпълнява финансови операции, преди отваряне на позиция се изискват няколко независими защитни проверки. Сигналят трябва да е времево валиден и да не е вече обработен. Конфликтни операции за един и същи bot, symbol и side трябва да бъдат сериализирани, а cooldown и bot-specific ограниченията да бъдат спазени.

Централният risk layer трябва да може да блокира нов entry при достигнати лимити за позиции, прекомерна експозиция, дневна загуба, drawdown или друго критично състояние. Целта му не е да определя дали пазарната прогноза е правилна, а да постави системна граница върху максимално допустимото действие.

Външното изпълнение изисква допълнителна защита. При timeout след изпращане на market order не е безопасно веднага да бъде изпратена идентична поръчка, защото първата може вече да е приета. Затова системата трябва да използва предвидими client order identifiers и последваща проверка на състоянието, вместо blind retry.

1.7. Среда за изпълнение и контролирано въвеждане

Платформата разграничава Paper, Demo и Production-ready режим. Paper средата симулира търговията вътре в системата и позволява да бъдат проверени signal processing, strategy, risk и persistence без външна поръчка. Demo средата използва реалните Binance transport механизми и UserStream събития, но без реални средства. Това я прави подходяща за проверка на signed requests, order lifecycle и reconciliation.

Production execution изисква най-строг контрол и не е необходима за доказване на техническата реализация в дипломната работа. По-важно е преминаването към нея да бъде съзнателна конфигурационна стъпка, а не неявно default поведение.

Освен execution environment системата разграничава и режими на генериране на сигнали. Това позволява вътрешна стратегия първо да бъде наблюдавана без реално изпълнение, а след това постепенно да бъде активирана в Paper или Demo. Така разработката поддържа контролирано преминаване от наблюдение към автоматизирано действие.

1.8. Изисквания към операторския интерфейс и обобщение

Операторският интерфейс осигурява централизирано наблюдение без директен достъп до PostgreSQL, Redis или Binance credentials. Чрез него потребителят следи runtime състоянието на ботовете, сигнали, позиции, Paper Trading, backtests, optimization, replay, health, alerts и audit данни. Write операциите се защитават чрез authentication, authorization, validation и idempotency.

От анализа следва, че системата не е само търговски алгоритъм, а платформа, която обединява market data, стратегии, risk, execution, persistence, analysis и operator control. Тези изисквания определят архитектурните решения в следващата глава.

ГЛАВА 2. АРХИТЕКТУРА НА СИСТЕМАТА

2.1. Архитектурен подход

Системата е изградена като набор от самостоятелни runtime приложения и споделени библиотеки, които следват естествените граници на търговския процес. Целта на това разделение не е използването на множество услуги само по себе си, а ограничаването на отговорностите и възможността отделните части да бъдат развивани, наблюдавани и тествани независимо.

Получаването на пазарни данни, изчисляването на индикатори, генерирането на сигнали, оценката на риска и изпълнението на поръчки имат различен lifecycle и различни failure сценарии. Dashboard също е отделен от runtime execution, защото операторският интерфейс не трябва да има пряк достъп до борсовите credentials, Redis state или вътрешните таблици.

В архитектурата могат да се разграничат три основни области: runtime trading pipeline, persistence и analytics слой, и control plane за управление и наблюдение. Тези области използват общи договори, но не споделят една обща синхронна транзакция от начало до край.

2.2. Основни компоненти и отговорности

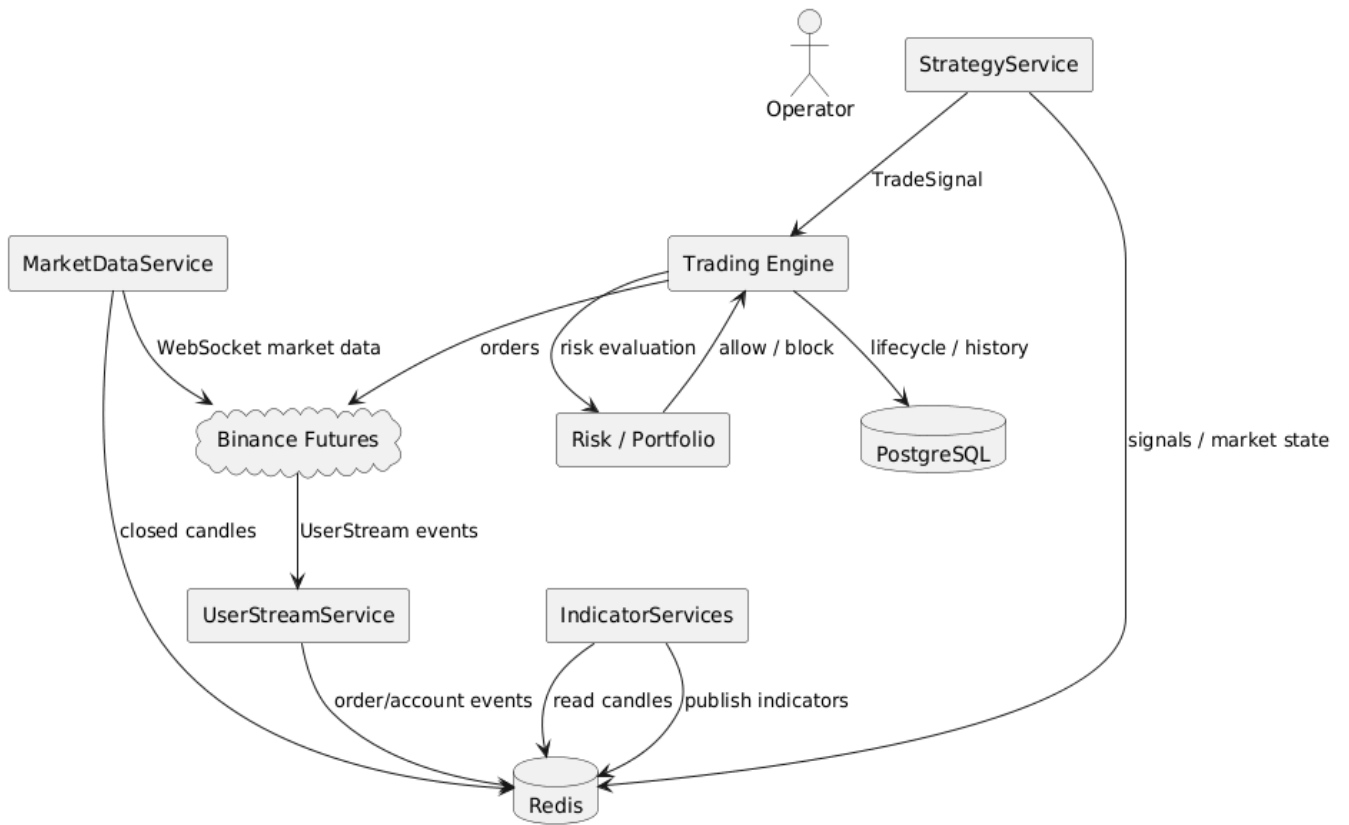
MarketDataService е входната точка за пазарните данни. Той поддържа връзка с Binance market stream, нормализира kline съобщенията и ги публикува към вътрешната комуникационна среда. IndicatorServices използва тези данни за изчисляване на технически индикатори, без да съдържа логика за отваряне на позиции.

StrategyService е отговорен за signal generation и за обработката на вече получени търговски сигнали. Тук се срещат bot-specific логиката и общият TradingEngine. TradingEngine не реализира конкретна стратегия, а организира последователността от общи safety проверки, strategy decision, risk evaluation и execution.

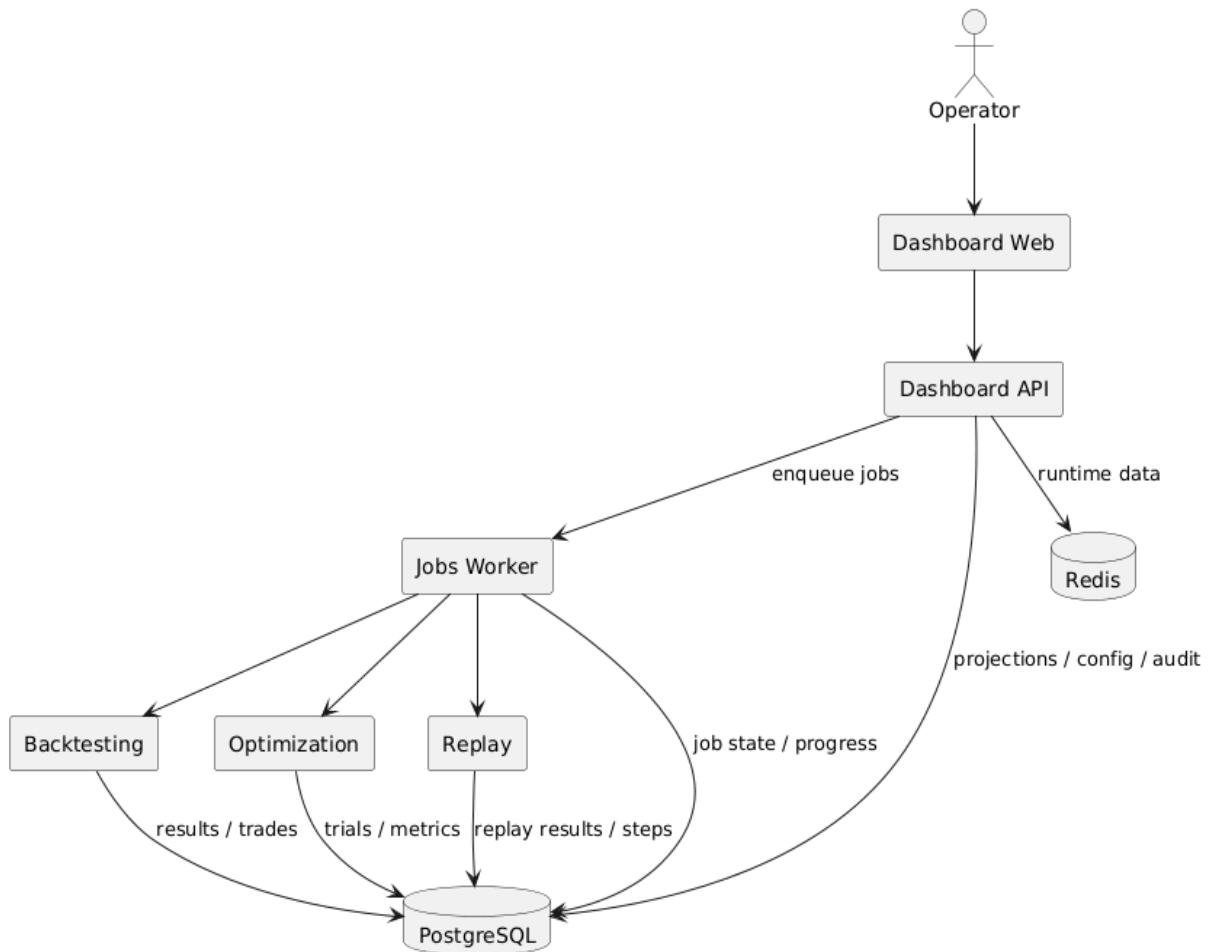
UserStreamService има обратна роля спрямо MarketDataService. Вместо публични пазарни данни той получава private order и account събития от Binance и ги предава към вътрешните lifecycle handlers. По този начин локалното състояние може да следва реалното изпълнение на поръчките, а не само първоначалния REST отговор.

Jobs Worker изпълнява продължителни background операции като historical ingestion, backtesting, optimization и replay. Dashboard API и Dashboard Web оформят отделен control plane, чрез който операторът наблюдава системата и стартира контролирани команди.

Φuzypa 2.1. Runtime trading flow



Φuzypa 2.2. Control plane, research u persistence



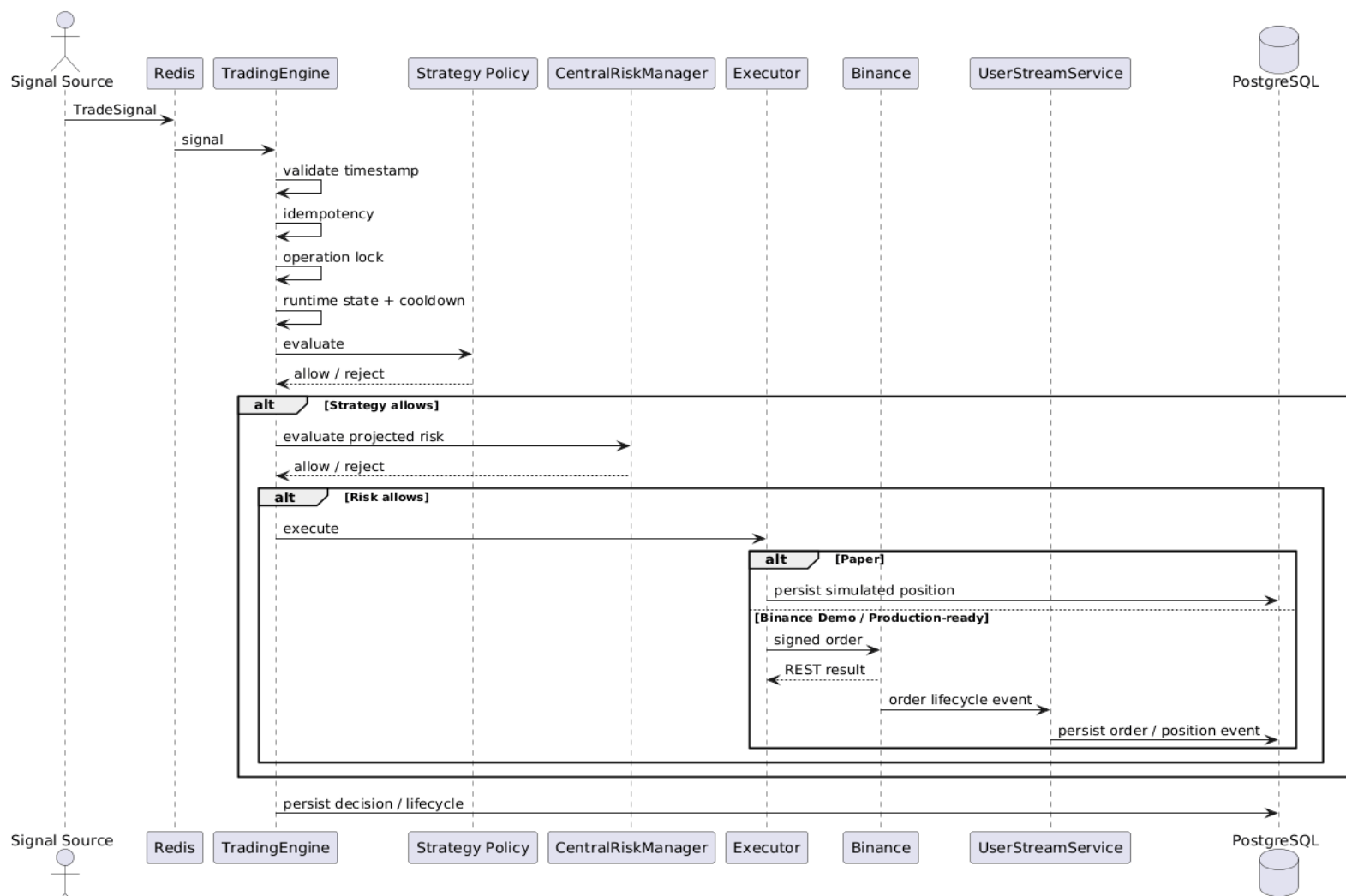
2.3. Runtime поток от пазарно събитие до изпълнение

Основният runtime поток започва с пазарна свещ. След нейното получаване и нормализиране тя може да бъде използвана от IndicatorServices и от стратегически компоненти. Когато условията на дадена стратегия са изпълнени, се създава TradeSignal или се приема такъв от външен източник.

Сигналът се публикува към общия канал за търговски сигнали и достига до TradingEngine. Преди изпълнение се проверяват валидността на сигнала, неговата идемпотентност, runtime състоянието на бота, конкурентният достъп, cooldown, bot-specific правилата и централният риск. Само положително решение на всички необходими нива позволява достигане до executor.

След външно изпълнение процесът продължава асинхронно. Binance може по-късно да изпрати fill, cancel или trigger събитие през UserStream. Следователно жизненият цикъл на една позиция не е единична HTTP операция, а последователност от събития, които могат да бъдат разделени във времето.

Фигура 2.3. Runtime поток от market event до position lifecycle



2.4. Комуникация чрез Redis

Redis се използва за бърза комуникация и краткотрайно runtime състояние. Пазарни свещи, индикаторни стойности, trading signals и определени вътрешни събития се разпространяват чрез Pub/Sub канали. Отделно Redis[2] се използва за coordination механизми като idempotency keys, operation locks, cooldown и distributed signal throttle.

Това решение е подходящо за hot path, защото предоставя ниска латентност и естествен TTL. Същевременно Pub/Sub не гарантира durable delivery. Поради това критичното бизнес състояние не трябва да съществува единствено като transient Redis message.

2.5. PostgreSQL и трайното състояние

PostgreSQL е трайното хранилище за данни, които трябва да останат достъпни след рестарт. Системата пази trading history, позиции, execution и order lifecycle информация, Paper state, Event Store, runtime и Dashboard projections, jobs, performance analytics, optimization и replay резултати, както и operational записи.

Разделянето на runtime communication от durable storage позволява всеки механизъм да бъде използван според силните му страни. Redis не се натоварва с дългосрочна история, а PostgreSQL не участва във всяка low-latency coordination проверка.

Трайният слой е важен и за възстановяването. След restart системата трябва да може да установи какво е било последното известно състояние и да го сравни с актуалните данни от Binance. Подробният модел на таблиците и аналитичните stores е разгледан по-късно отделно, за да не се смесва архитектурният преглед с persistence реализацията.

2.6. Strategy, Risk и Execution като отделни архитектурни граници

Strategy layer отговаря на въпроса дали при конкретното пазарно и позиционно състояние е допустимо ново действие. Risk layer отговаря на различен въпрос - дали цялата система може да си позволи допълнителната експозиция. Execution layer превежда одобреното бизнес решение към конкретна Paper или Binance операция.

Това разделение предотвратява ситуация, в която конкретен бот сам решава всички safety аспекти. Новата стратегия може да използва съществуващите правила за идемпотентност, runtime locking, cooldown и централизирана оценка на риска, без те да бъдат имплементирани повторно.

2.7. Външни и вътрешни източници на сигнали

Архитектурата не е ограничена до един източник на сигнали. Част от ботовете могат да използват външни сигнали, получени от TradingView, а други могат да генерират решения вътрешно върху пазарни и индикаторни данни.

Независимо от източника, след нормализиране сигналът използва общ TradeSignal contract и преминава през един и същ TradingEngine. Това е важно, защото TradingView сигнал не трябва да заобикаля risk или execution safety, а internal generator не трябва да получава привилегирован път до борсата.

Режимите на сигналите позволяват вътрешното поведение да бъде наблюдавано без изпълнение и постепенно да бъде преместено към реалния pipeline. Детайлите по webhook обработката, signal generation coordinator и режимите на изпълнение са разгледани в главата за реализация.

2.8. Paper, Demo и Production-ready execution

Execution средата е отделна от signal source и strategy type. Един вътрешно генериран сигнал може да бъде обработен в Paper среда, а TradingView сигнал може да достигне Demo executor. Това разделение позволява стратегията и safety pipeline да бъдат проверявани без да се променя тяхната логика.

Paper режимът завършва с локално симулирано изпълнение. Demo използва реалните signed Binance операции и UserStream събития, но без реални средства. Production използва същите инфраструктурни граници, но трябва да бъде активиран само чрез изрична конфигурация.

2.9. Reconciliation като архитектурен защитен слой

Локалната система и Binance са две независими системи и между тях няма обща транзакция. Поради network timeout, restart, пропуснато събитие или външна промяна може временно да възникне несъответствие между локалното и борсовото състояние.

Reconciliation съществува като отделен защитен слой, който сравнява двете гледни точки и регистрира отклоненията. Healing може да коригира само предварително дефинирани безопасни случаи. При неизвестно или рисково несъответствие архитектурата предпочита блокиране на нови действия и операторска намеса вместо автоматично предположение.

2.10. Research и аналитичен слой

Backtesting, optimization и replay са отделени от live execution, защото имат различен профил на натоварване и различна цел. Те могат да обработват големи исторически диапазони и множество trials, без да влияят на latency на runtime процесите.

Historical data предоставя входа за тези операции, а резултатите се съхраняват в отделни performance и research модели. Това позволява live trading и историческите експерименти да използват общи strategy policies, но да не споделят едно и също execution състояние.

2.11. Control plane и операторски интерфейс

Dashboard API и Dashboard Web са архитектурно отделени от trading runtime. Dashboard API предоставя read projections и контролирани write операции, докато Web приложението визуализира полученото състояние и не съдържа authoritative business logic.

По-дългите операции се моделират като jobs и се изпълняват от Jobs Worker. По този начин HTTP заявката не остава отворена по време на backtest, optimization или replay, а operator-ът може да следи lifecycle и резултата на задачата.

2.12. Observability и operational слой

Архитектурата включва observability като отделна отговорност. Healthcheck показва инфраструктурната готовност на даден container или endpoint, heartbeat показва дали приложението продължава активно да изпълнява своя lifecycle, а alerts и audit пазят трайна operational история.

Prometheus-compatible metrics допълват тази картина с количествени показатели. Така проблемът може да бъде разглеждан едновременно от гледна точка на наличност, runtime активност, performance и последователност на операторските действия.

2.13. Обобщение на архитектурните решения

Архитектурно решение	Основна причина
Отделен MarketDataService	Една точка за market stream и повторно използване на данните.

Отделен IndicatorServices	Индикаторната математика не се дублира във всяка стратегия.
Общ TradingEngine	Еднакъв safety pipeline независимо от bot и signal source.
Отделен Risk layer	Портфейлните ограничения не зависят от конкретната стратегия.
Paper/Binance execution boundary	Един и същ pipeline може да се проверява без реални средства.
Redis + PostgreSQL	Low-latency runtime state плюс durable history и analytics.
UserStream + reconciliation	Локалното състояние се сверява с реалните борсови събития.
Jobs Worker	Дългите операции не блокират HTTP lifecycle.
Dashboard като control plane	UI наблюдава и управлява системата без директен достъп до execution слоя.

ГЛАВА 3. ТЕХНОЛОГИЧНА ОСНОВА И ИНЖЕНЕРНИ РЕШЕНИЯ

3.1. Критерии при избора на технологичния стек

Технологичният стек е избран според характера на системата: непрекъснати връзки към външна борса, асинхронна обработка на събития, ниска латентност при runtime проверки, трайно съхраняване на история и отделен операторски интерфейс. Целта не е използване на максимален брой технологии, а всяка технология да има ясна роля и обосновано място в архитектурата.

Основните критерии са надеждност, тестируемост, предвидимо поведение при грешка, възможност за контейнеризирано разгръщане и лесно наблюдение. Компонент, който участва в critical execution path, трябва да има ясни failure semantics. Компонент, който е част от analytics или UI, трябва да може да се развива без да променя trading execution.

Отделянето на runtime communication от durable storage е пряко следствие от тези критерии. Redis се използва там, където е необходим бърз transient state, а PostgreSQL - когато информацията трябва да преживява restart и да участва в последващи заявки, анализ и audit.

3.2. C# и .NET 9 като backend основа

Backend компонентите са реализирани на C# върху .NET 9[12]. Платформата е подходяща за дълго работещи услуги с множество асинхронни I/O операции към Redis, PostgreSQL, Binance REST API и WebSocket потоци. async/await моделът позволява тези операции да бъдат изпълнявани без блокиране на worker threads при изчакване на мрежов отговор.

Dependency Injection се използва като основен механизъм за свързване на application и infrastructure слоевете. TradingEngine работи през интерфейси за risk, execution, state и history и не е свързан директно с конкретен Binance client или PostgreSQL repository. Това позволява същата orchestration логика да се използва с Paper executor и с Demo executor.

Options pattern предоставя strongly typed configuration. Bot-specific параметри, risk лимити, endpoint настройки и worker интервали се свързват към типизирани класове и могат да бъдат валидирани при startup. Така конфигурационна грешка се открива преди да достигне до trading path.

3.3. BackgroundService и lifecycle на runtime услугите

MarketDataService, IndicatorServices, StrategyService, UserStreamService и Jobs Worker съдържат дълго работещи background компоненти. Те използват BackgroundService и CancellationToken, за да могат да стартират, да работят непрекъснато и да приключват контролирано при stop на container.

Контролираното прекратяване е важно при event-driven система. Host cancellation не трябва да бъде записана като trading failure, а като нормален lifecycle на process. По същия начин subscription или WebSocket loop трябва да освобождава ресурсите си в finally блок и да не оставя частично състояние.

PeriodicTimer се използва при периодични задачи като refresh на конфигурация, heartbeat и reconciliation. Това е по-предвидимо от безкраен цикъл с несвързани Task.Delay извиквания и улеснява коректната cancellation семантика.

3.4. ASP.NET Core и application boundary

ASP.NET Core/[13] се използва за Dashboard API и TradingView webhook входа. HTTP слой е граница между външния клиент и application логиката. Controller или minimal endpoint не трябва директно да изпълнява SQL или signed Binance операция без междинен application contract.

Dashboard API разделя read и write сценарии. Read операциите зареждат projections, докато write операциите създават commands или jobs. Това е важно при дълги операции като optimization и replay: HTTP request приключва бързо, а Jobs Worker поема фактическата обработка.

Global exception handling преобразува известни проблеми към контролирани HTTP responses, а correlation identifier позволява конкретна заявка да бъде намерена в логовете.

3.5. Authentication, authorization и сигурност на операторския интерфейс

Операторският интерфейс изисква authentication преди достъп до trading projections и control actions. Паролата се съхранява като PBKDF2-SHA256 hash, а след успешно вписване се издава token с ограничен lifetime.

Authorization разделя правата за наблюдение и действие. Viewer може да чете projections, докато Operator или Administrator изпълнява configuration changes, runtime commands и job creation. По-рисковите операции използват допълнително потвърждение и по-строго rate limiting.

Конфигурационните secrets се подават чрез environment variables и .env файл. При deployment върху NAS това позволява image-ът да остане без вградени API keys и passwords.

3.6. Redis като communication и coordination слой

Redis има две различни роли. Първата е Pub/Sub комуникация за market data, indicator snapshots, trading signals и вътрешни събития. Втората е coordination state: idempotency keys, operation locks, cooldown, throttle и position-related runtime indexes.

Тези два вида употреба имат различни характеристики. Pub/Sub е подходящ за low-latency transient distribution, но не пази съобщение за consumer, който е offline. Coordination записите от своя страна използват TTL и атомарни операции, за да ограничат времето на lock или cooldown.

3.7. PostgreSQL, Npgsql и Dapper

PostgreSQL/[14] е основното трайно хранилище. В него се пазят trading history, Event Store, Paper state, runtime configuration, jobs, performance analytics, optimization, replay, heartbeats, alerts и audit. Тези данни имат различни query patterns и затова са организирани логически по schemas.

Npgsql предоставя .NET driver за PostgreSQL и се използва за connection management и async database access. Dapper се използва при read-oriented и аналитични заявки, където е необходим директен контрол върху SQL, joins, pagination и projections. Използването на explicit SQL улеснява изграждането на Dashboard views и агрегати, но изисква дисциплина при schema evolution.

3.8. SQL migrations и versioning на схемата

Database schema се развива чрез последователни migration файлове. В текущата реализация те покриват trading history, analytics, risk/reconciliation, dashboard backend, runtime orchestration, jobs, service health, Paper Trading, strategy plugin system, replay, Event Store и reliability hardening.

Migration tracking записва кои версии са приложени и позволява checksum проверка. Това предотвратява неясна промяна на вече изпълнена migration и прави възстановяването на нова среда повторимо.

3.9. Binance Futures REST API/1/

REST API се използва за explicit actions: account state, leverage, create/cancel/query на поръчки и други signed операции. Signed request изисква API credentials, timestamp и signature, поради което тази логика е изолирана в infrastructure слой.

При trading execution request timeout не трябва автоматично да се интерпретира като failure. Възможно е Binance да е приел поръчката, но отговорът да не е достигнал до приложението. Затова се използват предвидими client order identifiers и последващ lookup преди евентуално повторение.

Demo endpoint позволява същият transport и signing механизъм да бъде проверен без реални средства. Това е причината Binance Demo да е част от верификационния процес преди production-ready режим.

3.10. WebSocket потоци: MarketData и UserStream

Public market stream и private UserStream имат различна семантика. MarketDataService получава kline данни и ги нормализира за вътрешна употреба. UserStreamService получава събития за поръчки и account state и ги използва за актуализиране на lifecycle след REST execution.

WebSocket връзките трябва да поддържат reconnect и freshness checks. При market data е важно да се работи със затворени candles, когато стратегията изисква окончателни стойности. При UserStream е важно duplicate event да не води до повторен lifecycle transition.

Комбинацията REST + WebSocket отразява реалната природа на exchange integration: REST инициира действие, а stream-ът доставя последващи факти за неговото изпълнение.

3.11. React, TypeScript и TanStack Query

Dashboard Web е реализиран с React[16] и TypeScript[18]. Feature-oriented структурата разделя Bots, Signals, Positions, Trades, Analytics, Backtests, Optimization, Replays и Operations, вместо всички компоненти да бъдат концентрирани в една страница.

TypeScript дава типизирани API models и намалява риска frontend да интерпретира погрешно enum или nullable поле. TanStack Query[17] управлява server state, caching, polling, refetch и invalidation. След command или configuration update frontend не приема локално, че действието е завършило, а презарежда authoritative projection от backend.

Production build-ът се обслужва от Nginx. Той сервира статичните assets и проху-ва /api/ към Dashboard API, което скрива вътрешните service addresses от browser клиента.

3.12. Docker и Docker Compose

Docker се използва за стартиране на отделните компоненти на системата в изолирана и предвидима среда. Това улеснява локалната разработка, тъй като услугите работят с еднакви зависимости и настройки независимо от конкретната машина.

Docker Compose[15] обединява конфигурацията на приложенията и инфраструктурните услуги в един общ файл. Чрез него се задават необходимите контейнери, мрежи, портове, persistent volumes и environment variables, което прави стартирането на цялата система по-лесно и повторимо.

3.13. Контейнеризирано разгръщане върху Synology NAS

За практическата демонстрация системата се стартира и върху Synology NAS. Това позволява тя да работи независимо от development машината и да бъде достъпна като постоянно работеща среда. Използването на NAS дава възможност да се наблюдава поведението на контейнерите при

по-продължителна работа, както и съхранението на данните, мрежовата свързаност и състоянието на отделните услуги.

3.14. Наблюдаемост: health, heartbeat, metrics и notifications

Наблюдаемостта е реализирана на няколко нива. Container healthcheck показва дали endpoint е достъпен. Application heartbeat показва кога услугата последно е потвърдила работата си. Alerts пазят значими operational проблеми, а metrics позволяват количествен анализ.

Prometheus-compatible метриките могат да измерват request count, duration, errors, process memory и специфични processing показатели. Те допълват логовете, защото позволяват тенденции и агрегати за период от време.

Telegram notifications се използват за бърза информация при position opened или execution failure. Те са non-critical side effect: ако notification не успее, потвърденото trading state не трябва да бъде връщано назад.

3.15. xUnit тестове

За част от .NET компонентите в системата се използва xUnit[19]. Чрез него са реализирани unit тестове за отделни части от бизнес логиката. Тестовите използват dependency injection и mock зависимости, когато е необходимо конкретен компонент да бъде проверен без реална връзка към Redis, PostgreSQL или Binance.

3.16. Основни технологични компромиси

Избраните технологии дават ясни предимства, но всяка носи и ограничение. Redis Pub/Sub е бърз, но не е durable. PostgreSQL е надежден за history и analytics, но не трябва да участва във всеки low-latency lock check. REST е подходящ за commands, но timeout може да остави uncertain outcome. WebSocket дава навременни събития, но изисква reconnect и deduplication.

Dapper[20] осигурява контрол върху SQL и нисък overhead, но изисква повече ръчен mapping. Docker Compose прави deployment-а повторям, но увеличава operational сложността при множество services. React Query опростява server-state lifecycle, но изисква дисциплина при query keys и invalidation. Тези компромиси са приети съзнателно, защото отговарят на конкретните граници на системата. Няма една технология, която едновременно да осигури low latency, durable delivery, analytics, UI и exchange integration.

ГЛАВА 4. РЕАЛИЗАЦИЯ НА ОСНОВНИТЕ МЕХАНИЗМИ НА СИСТЕМАТА

4.1. Организация на реализацията

Практическата реализация е организирана така, че конкретната стратегия да не носи отговорност за всички инфраструктурни и защитни механизми. Bot-specific кодът определя кога е допустим входът и как се управлява позицията, докато общите application компоненти осигуряват идемпотентност, runtime synchronization, cooldown, risk evaluation, execution safety, event history и recovery.

Това разделение позволява една и съща инфраструктура да се използва от различни стратегии и от различни източници на сигнали. По този начин TradingView сигнал, вътрешно генериран сигнал и контролиран Paper събитие преминават през един и същ TradingEngine, вместо да имат отделни и несъвместими execution пътища.

4.2. TradingView като външен източник на сигнали

TradingView се използва като външен източник на търговски сигнали. Alert от TradingView достига до webhook endpoint, където входният payload се валидира и се преобразува към вътрешния TradeSignal contract. Така външният формат не се разпространява навътре в системата и останалите компоненти не зависят от конкретния webhook модел.

След нормализиране сигналът съдържа SignalId, bot, symbol, side, source, strategy version и timestamp. Source позволява по-късно да се различи външният TradingView сигнал от вътрешно генериран candidate, но след тази точка двата вида събития използват общ processing pipeline.

Това е важна safety граница: webhook-ът не изпраща поръчка директно към Binance. Той само създава формализирано намерение за действие. Реалното решение за изпълнение остава в TradingEngine, strategy policy и CentralRiskManager.

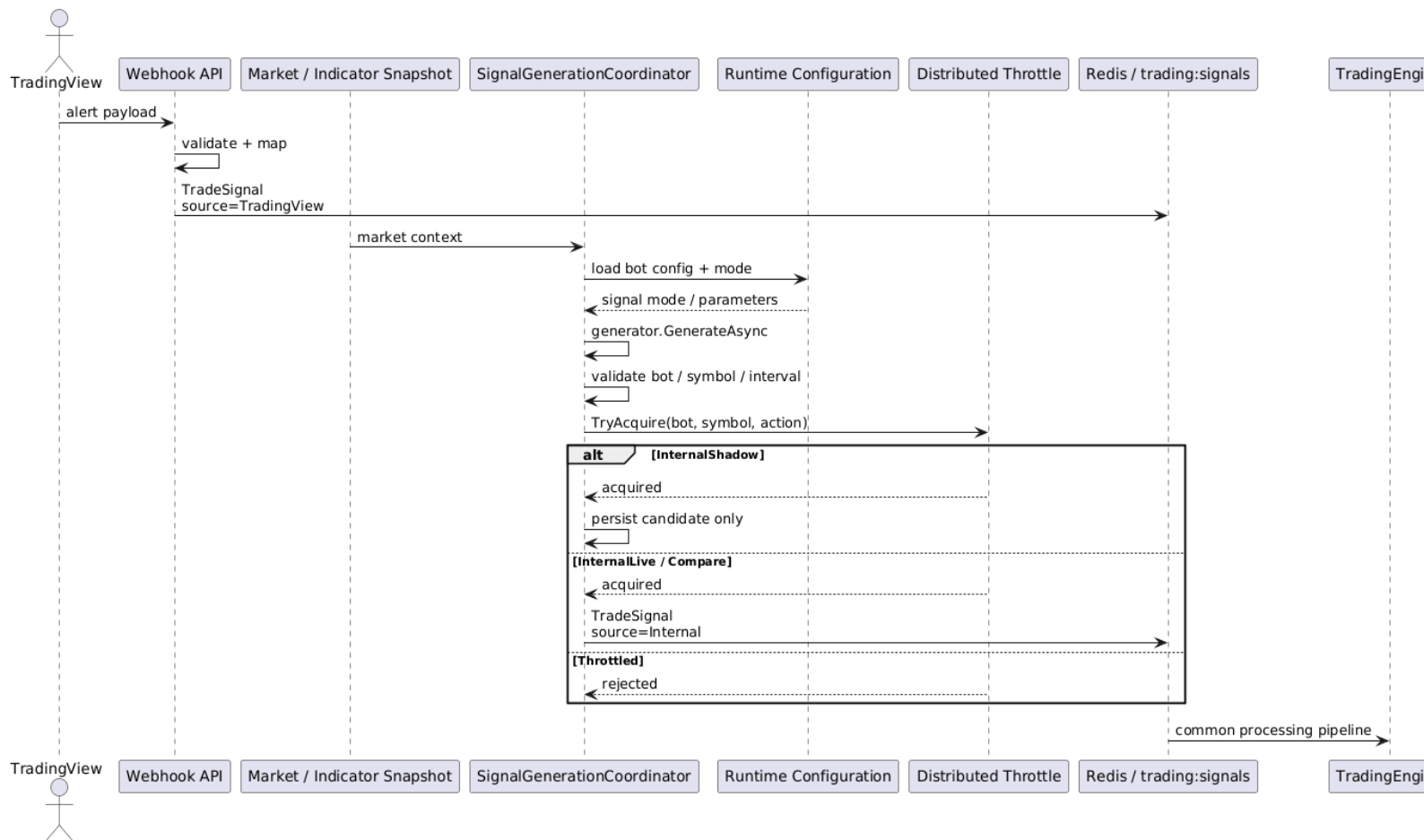
4.3. Вътрешно генериране на сигнали

Вътрешното генериране започва от пазарни и индикаторни snapshots. SignalGenerationCoordinator избира generator-ите, приложими за конкретния symbol, зарежда runtime configuration на съответния bot и проверява signal mode.

След извикване на generator-а полученият candidate се валидира спрямо обработвания market context. Bot, symbol и interval не трябва да се разминават със snapshot-а, върху който е взето решението. Преди публикуване се прилага distributed throttle за комбинацията bot, symbol и action. Ако минималният интервал между два вътрешно генерирани сигнала не е изтекъл, новият candidate не достига до `trading:signals`.

Режимът определя какво се случва след генерирането. При TradingViewOnly вътрешният generator не участва в trading decisions. InternalShadow позволява сигналът да бъде генериран и записан, но не и изпълнен, а InternalLive публикува валидния сигнал към общия pipeline.

Фигура 4.1. Външно и вътрешно генериране на TradeSignal



4.4. Runtime конфигурация и кеширане

Поведението на един bot не се определя единствено от статичен `appsettings.json`. Runtime конфигурацията съдържа параметри като environment, signal source, enable/disable на LONG и SHORT, quantity, leverage, cooldown и bot-specific стойности.

`CachedBotRuntimeConfigurationProvider` отделя persistent конфигурацията от hot path. TradingEngine и signal generation компонентите могат да използват актуално runtime състояние, без при всяко пазарно събитие да изпълняват SQL заявка. Това е важно при чест поток от candles и signals, където database round-trip за всяка проверка би бил ненужен overhead.

Runtime status и execution enabled са отделни от strategy configuration. По този начин bot може да бъде Started, Paused или Stopped без промяна на самата стратегия. Същият механизъм позволява Paper, Demo и Production-ready execution да бъдат сменяни контролирано чрез конфигурацията, а не чрез различен кодов път.

4.5. TradingEngine и последователност на защитните проверки

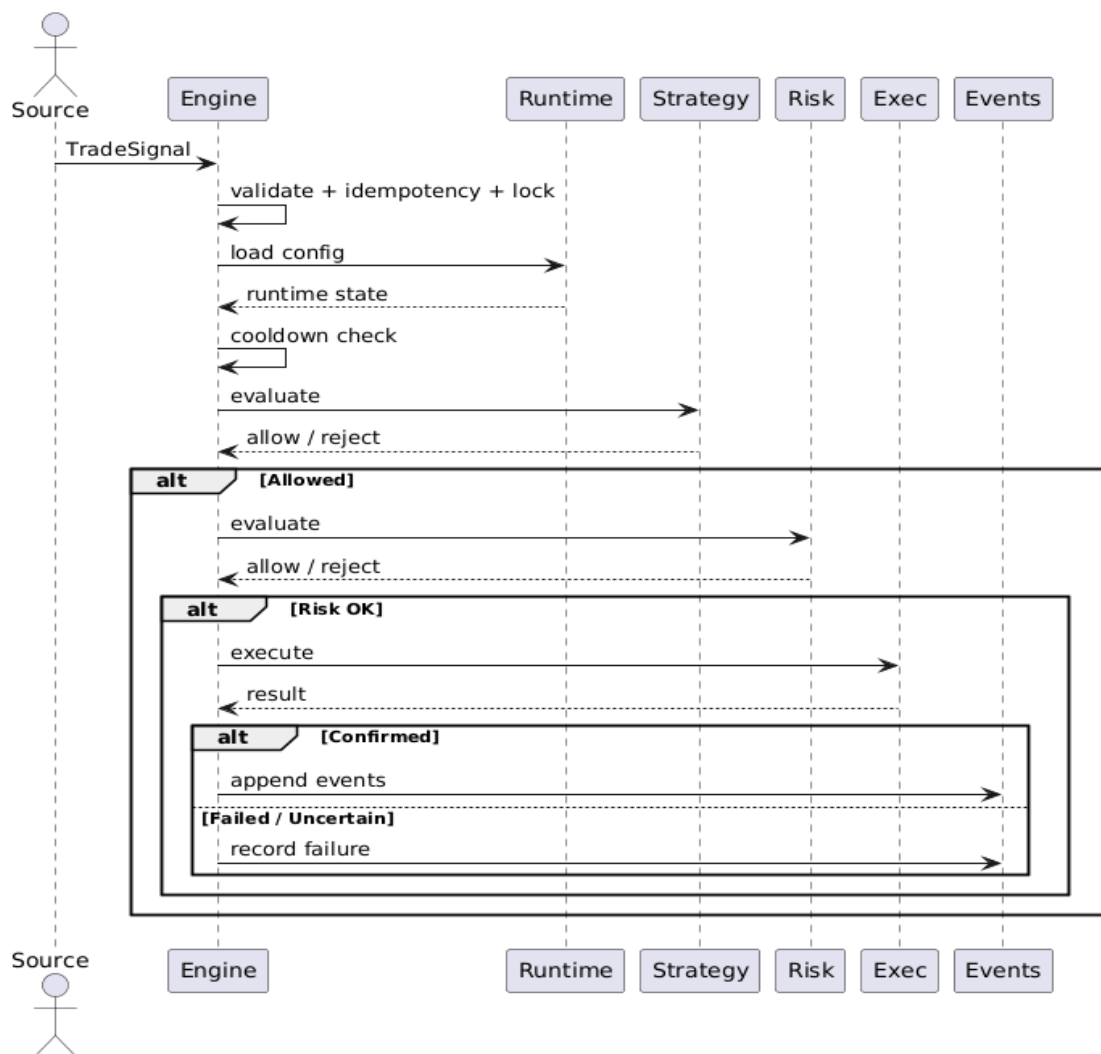
TradingEngine е orchestration границата между TradeSignal и реалното изпълнение. Сигналят първо се проверява за валиден timestamp и за повторна обработка. След това се придобива distributed operation lock за конкретната комбинация bot, symbol и side.

След lock-а се зареждат runtime state и конфигурацията, проверяват се execution enabled и cooldown и едва тогава се извиква конкретната strategy policy. При положително strategy решение се оценява

central risk върху текущото и projected portfolio state. При необходимост се използва risk reservation, за да не могат две конкурентни операции едновременно да използват един и същ свободен лимит.

Едва след всички тези проверки се извиква executor. След потвърдено execution състоянието на signal idempotency не трябва да се освобождава дори ако по-късно се провали notification или bookkeeping операция. Това разграничение е критично: повторен сигнал след вече отворена позиция би могъл да създаде двойна експозиция.

Фигура 4.2. Последователност в TradingEngine и граница на критичния execution path



4.6. Идемпотентност, operation lock, cooldown и signal throttle

Четири механизма решават различни проблеми. Idempotency предотвратява повторно логическо изпълнение на един и същ SignalId. Operation lock сериализира конфликтни операции за един bot, symbol и side. Cooldown ограничава колко често вече приет сигнал може да доведе до нова търговска операция.

Signal throttle се прилага по-рано - при генериране на вътрешни сигнали. Той не е transport rate limit и не защитава външен endpoint. Целта му е два близки market snapshots да не публикуват множество еквивалентни internal candidates преди те изобщо да достигнат TradingEngine.

4.7. BOT8012, BOT8013 и BOT8014 - TP-only Grid

BOT8012, BOT8013 и BOT8014 използват общ модел от тип TP-only Grid. Основната идея е нова позиция да се отваря само когато има достатъчно ценово отстояние спрямо съществуващата grid структура. Конфигурацията включва Quantity, Leverage, PriceDistance, ProfitDistance, OrderSideLimit и CooldownSeconds.

При сигнал се проверяват активните позиции в същата посока. Ако е достигнат OrderSideLimit, нов вход не се допуска. В противен случай GapPolicy сравнява текущата цена с релевантната най-нова позиция, като условието е огледално за LONG и SHORT.

След положително решение executor-ът отваря позицията и създава take-profit според ProfitDistance. Общите TpOnlyGrid компоненти позволяват трите бота да използват еднаква базова логика, а healing механизмът възстановява липсващ локален state от наличните Binance orders, когато това е безопасно.

4.8. BOT8011 и BOT8015 - ProtectedStop3

BOT8011 и BOT8015 използват по-сложен lifecycle. След market entry се създават take-profit и stop-loss поръчки, а take-profit може да затвори само част от първоначалното количество. Оставашата част преминава към следващ защитен етап.

При този модел не е достатъчно позицията да е само Open или Closed. Трябва да се проследи дали първоначалният TP е изпълнен, какво е remaining quantity, дали началният SL е отменен и дали е създадена STOP3 защита.

Partial fill идва като UserStream event. Handler-ът актуализира lifecycle състоянието и задейства следващите действия. Ако STOP3 creation се провали след реално partial изпълнение, операцията не може да бъде върната назад като database transaction. Затова подобни случаи изискват explicit state, alert и reconciliation.

4.9. BOT8016 - Alligator Multi-Timeframe

BOT8016 използва различен модел, основан на технически индикатори и повече от един времеви интервал. Entry логиката използва Alligator и пресичане на цената спрямо Teeth, като по избор може да се приложи SMA200 филтър.

Entry candle и indicator state трябва да бъдат съпоставени коректно по CandleCloseTime, за да не се използва бъдеща информация. Exit логиката може да работи върху по-кратък interval и да използва последната валидна стойност от по-големия timeframe.

След partial take-profit могат да се активират допълнителни защитни правила и последващ exit. Тази стратегия показва защо timestamp correctness и freshness са част от domain поведението, а не само техническа подробност.

4.10. Paper Trading като реален execution режим

Paper Trading не е отделна опростена стратегия. Сигналът преминава през същите проверки за idempotency, operation lock, runtime state, cooldown, strategy policy и CentralRiskManager, след което вместо Binance executor се използва simulated execution.

Paper позицията има собствен lifecycle и съдържа entry price, quantity, side, както и assumptions за fees и slippage. Следващите market data събития могат да доведат до take-profit, stop-loss или strategy exit според модела на конкретния bot. Това позволява Paper средата да се използва като истински end-to-end тест на application pipeline.

4.11. Binance execution и uncertain outcome

При Demo или Production-ready режим executor-ът превежда одобреното решение към signed Binance REST операции. Освен entry параметрите могат да бъдат създавани bot-specific protective orders.

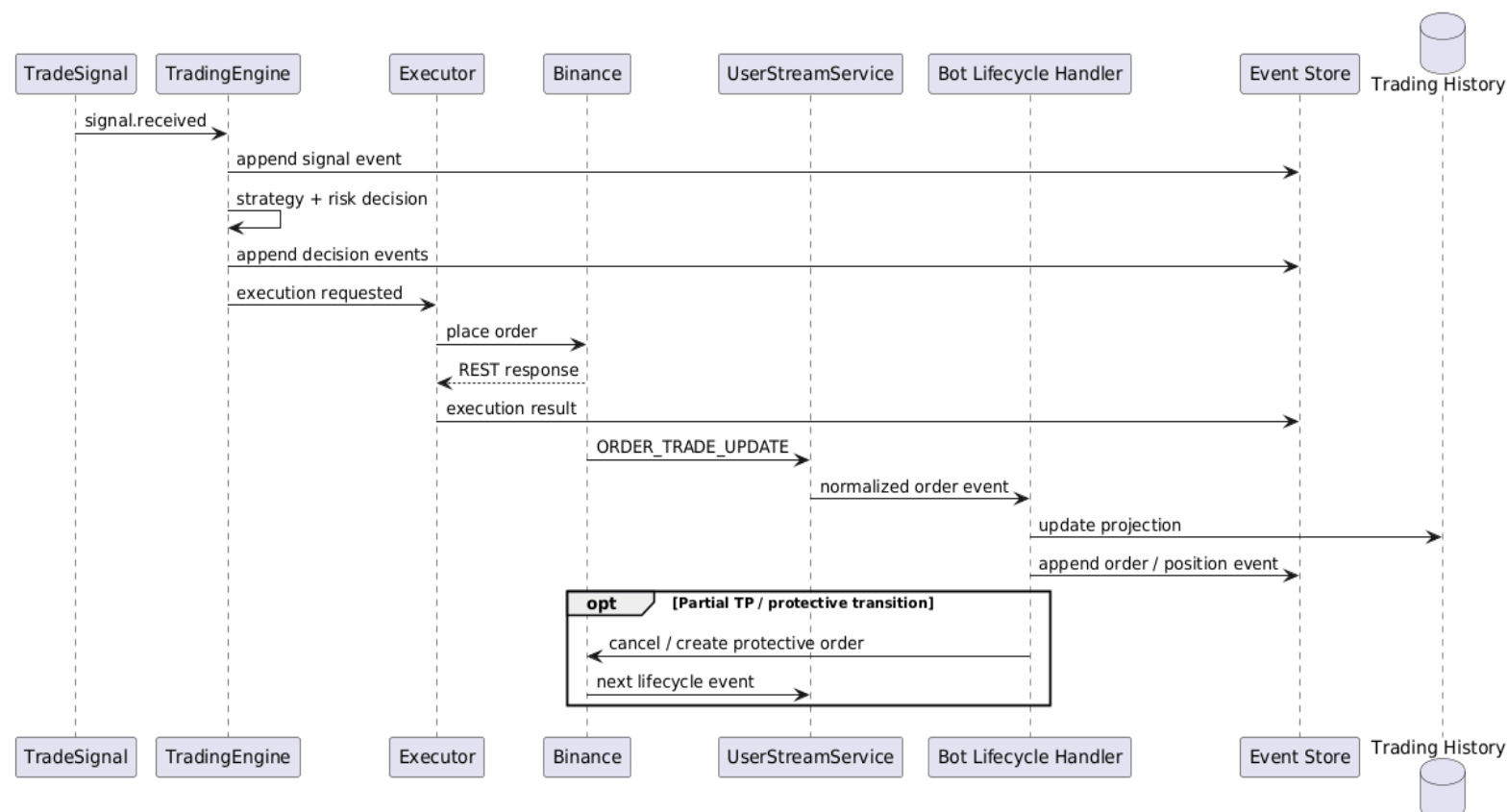
Критичният failure сценарий е uncertain outcome. Ако request е изпратен, но response не бъде получен поради timeout, не е безопасно да се приеме, че поръчката не съществува. Blind retry на market order може да отвори втора позиция. Затова външните операции използват предвидими client order identifiers. След timeout системата може да провери дали конкретната операция вече съществува и да класифицира резултата като confirmed success, definite failure или uncertain state, изискващо последваща проверка.

4.12. UserStream и последователността на lifecycle събитията

REST response описва само непосредствения резултат от дадена заявка. Реалният lifecycle на поръчката продължава през UserStream. Поръчка може по-късно да бъде NEW, PARTIALLY_FILLED, FILLED, CANCELED или задействана като защитна поръчка. UserStreamService преобразува външното Binance събитие към вътрешен contract и го публикува към bot-specific handlers. Тези handlers актуализират локалната позиция, remaining quantity и protective state и могат да породят следващо действие, например създаване на STOP3 след partial TP.

На логическо ниво последователността може да се разглежда като: signal received -> strategy decision -> risk decision -> execution requested -> order accepted -> order event -> position state change -> performance/event history. Event Store пази значимите факти по този lifecycle, така че при troubleshooting да може да се възстанови как системата е стигнала до него.

Фигура 4.3. Последователност от TradeSignal до UserStream lifecycle и Event Store



4.13. Retry, permanent failure и странични ефекти

Retry не се прилага еднакво към всички грешки. Временна недостъпност на инфраструктура или background job може да бъде повторена след интервал, но permanent failure не трябва да създава безкрайни опити.

При Jobs Worker attempt_count се увеличава при claim. Неуспешната задача може да бъде върната в Pending със следващо време за опит, докато се достигне максималният брой. След това се маркира Failed и се записва worker-ът, приключил последния attempt.

При борсово изпълнение retry е по-консервативен, защото грешката може да е настъпила след реален side effect. Ако exchange execution вече е потвърдено, последващ проблем в notification или bookkeeping не трябва да освобождава signal idempotency и да представя операцията като безопасна за повторение.

Telegram е пример за non-critical side effect. Неуспешно известие се логва като warning, но не отменя успешно създадена позиция. Host cancellation също се различава от trading failure и трябва да бъде оставена да прекъсне background lifecycle коректно.

4.14. Reconciliation и healing

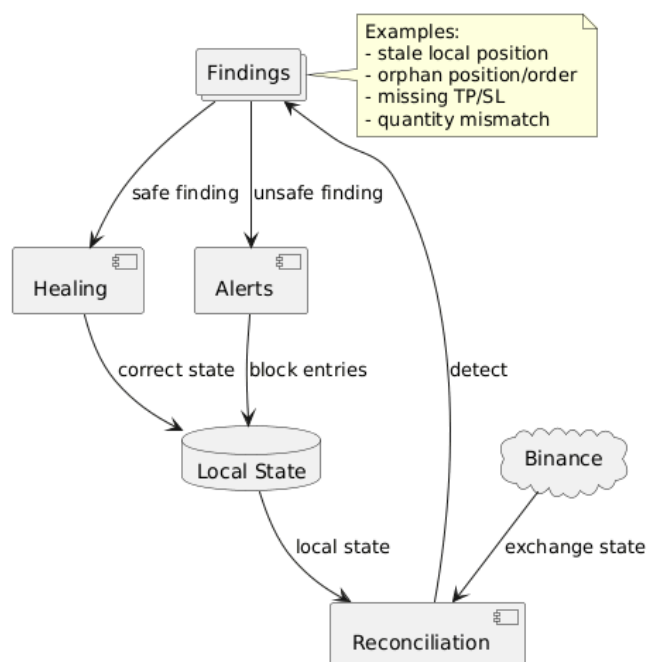
Reconciliation сравнява локалното състояние с authoritative данните от Binance. Причина за разминаване може да бъде restart, прекъснат UserStream, timeout след успешно приета поръчка, ръчна промяна в Binance или частично изпълнение.

Проверките могат да открият stale local position, orphan exchange position, orphan exchange order, липсващ take-profit или stop-loss и quantity mismatch. При Hedge Mode е важно exchange позицията да се разглежда агрегирано по symbol и side, защото Binance не връща отделна position-risk позиция за всеки bot.

Detect и Heal са разделени. Първо се създава списък с findings и тяхната severity. Само предварително определени безопасни случаи могат да бъдат auto-heal-нати. Например stale local state може да бъде изчистен автоматично при достатъчно доказателства, докато липсваща protective order обикновено изисква по-консервативно поведение.

Това разграничение запазва audit trail и предотвратява скрити промени върху exchange state. При неясно несъответствие системата предпочита fail-closed поведение - блокиране на нови entries и операторска проверка.

Фигура 4.4. Reconciliation: detect, classify u controlled healing



4.15. Telegram и operational notifications

Telegram notification слойът предоставя бърза обратна връзка за значими execution събития. Съобщения могат да се изпращат при отворена Paper или Binance позиция, при execution failure и при други важни lifecycle промени. Известяването е умишлено отделено от критичния execution path. Ако Telegram е временно недостъпен, системата трябва да запази вярното състояние на позицията и да логне проблема, вместо да маркира търговската операция като неуспешна.

4.16. Event Store и проследимост

Event Store пази append-only история на значимите факти в търговския lifecycle. Записите включват идентификатори за event, aggregate, correlation и causation, както и bot, signal или position context. Тази история има различна роля от текущата projection. Projection отговаря на въпроса какво е състоянието сега, а Event Store позволява да се проследи как то е достигнато. При проверка за идемпотентност целта не е само да има една database position. За един логически signal lifecycle трябва да съществуват точно очакваните събития, без duplicate `signal.received` или `position.opened` записи.

4.17. Обобщение на failure поведението

Ситуация	Поведение	Причина
Duplicate SignalId	Не се изпълнява повторно	Логическа идемпотентност
Конкурентен сигнал	Operation lock сериализира	Защита от race condition
Internal signal твърде скоро	Throttle блокира publication	Ограничение преди TradingEngine
Сигнал в cooldown	Execution се отказва	Временна защита след предходно действие

Binance definite reject	Операцията приключва като failure	Няма потвърден exchange side effect
Binance timeout	Lookup / reconciliation	Възможен uncertain outcome
Telegram failure	Warning, без rollback	Notification е non-critical side effect
Job transient failure	Retry до max attempts	Background recovery
Unsafe reconciliation finding	Block / operator review	Fail-closed поведение

ГЛАВА 5. PERSISTENCE, АНАЛИЗ, ОПТИМИЗАЦИЯ И ОПЕРАТОРСКИ КОНТРОЛ

5.1. Роля на persistence слоя

В разработената система PostgreSQL не е просто хранилище за крайния резултат от една сделка. Базата участва в няколко независими процеса: съхраняване на търговска история, runtime конфигурация, Paper Trading state, Event Store, jobs, performance analytics, optimization, replay, service health, alerts и audit. Общото между тези данни е, че трябва да преживяват рестарт на отделна услуга и да бъдат достъпни за последващ анализ.

Логическото разделение по schemas позволява различните модели да се развиват независимо. Данните за runtime търговия имат различна честота на запис и различни заявки от тези за optimization trials или service heartbeats. Това е причина да не се използва една универсална таблица с динамичен payload, а по-ясни модели с конкретна роля. Така query слойът може да бъде оптимизиран за реалните сценарии на Dashboard, а аналитичните операции не натоварват излишно оперативните таблици.

Persistence слойът е и важна част от recovery модела. Redis съдържа бързо и краткотрайно състояние, но при restart приложението трябва да може да възстанови контекста си от трайни записи и да го сравни с Binance. По тази причина локалната позиция, order events, runtime state, reconciliation findings и event history имат различна, но допълваща се роля.

5.2. Trading history и проследимост на една търговска операция

Trading history описва системата не само като набор от отворени и затворени позиции, а като последователност от входни събития и решения. Записът на TradeSignal съдържа идентификатор, bot, symbol, side, source и timestamp. След него strategy decision показва дали конкретната стратегия е разрешила действие и по каква причина. Position и order events описват какво действително се е случило след execution.

Това разделение е съществено при диагностика. Например сигнал може да бъде приет, но да бъде блокиран от risk manager, или стратегията да реши, че ценовото отстояние е недостатъчно. В друг случай execution може да е успешно, но последващ UserStream event да покаже cancel или частично изпълнение. Ако се пази само крайната позиция, тези разлики се губят.

History моделът позволява и проверка на идемпотентността. При повторен SignalId очакваният резултат е да няма втори логически execution lifecycle. Следователно диагностиката не се ограничава до броя позиции, а включва и броя на събитията signal.received, strategy decision и position.opened за една и съща корелация.

5.3. Event Store, projections и причинно-следствена връзка

Event Store пази append-only факти с глобална последователност и идентификатори за correlation и causation. За разлика от projection таблиците, които са създадени за удобни заявки, Event Store цели да запази причинно-следствената история. Projection отговаря на въпроса „какво е текущото състояние“, докато Event Store позволява да се отговори „как системата е стигнала до него“.

Това разграничение има практическа стойност при несъответствие. Ако Binance вече показва нулева експозиция, а локална projection все още показва Open, projection може да бъде коригирана или преизчислена. Event Store не трябва да се редактира, защото съдържа записаните факти за signal, execution и последващите lifecycle събития.

Replay използва именно стабилната последователност от събития. При еднакъв диапазон и еднакъв ред на входните записи детерминистичният hash трябва да бъде повторям. Така Event Store е не само audit механизъм, а и техническа основа за regression анализ.

5.4. Paper Trading state и изолация на тестовата среда

Paper Trading използва отделно persistence състояние, защото simulated позиция не трябва да се смесва с exchange позиция. Това е важно както за Dashboard, така и за risk и reconciliation. Binance snapshot не трябва да бъде сравняван с Paper exposure, а Paper reset не трябва да променя Demo история.

Paper режимът е проектиран като реален execution boundary. До него сигнала преминава през същите общи проверки за runtime status, idempotency, operation lock, cooldown, strategy policy и CentralRiskManager. Разликата е в executor-а. Това позволява да се проверява целият application pipeline без външна финансова операция.

Изолираната Paper persistence структура улеснява и тестовите на lifecycle. Може да се създаде контролирана simulated position, да се приложат market data събития и да се провери TP, SL или strategy exit, без да се разчита на движението на реалния пазар.

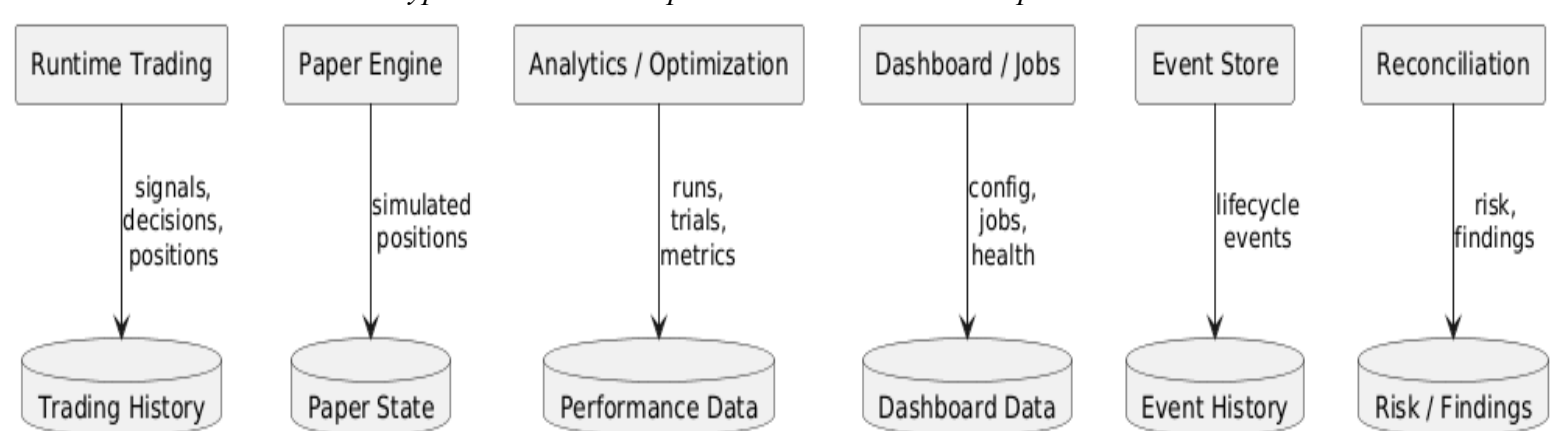
5.5. Risk, reconciliation и operational данни

Рискът и reconciliation имат отделни трайни записи, защото описват причините системата да позволи, блокира или постави под съмнение дадено действие. Risk event може да съдържа код като MAX_OPEN_POSITIONS или DAILY_DRAWDOWN_LIMIT и да обясни защо execution не е достигнат.

Reconciliation run съдържа информация кога е извършено сравнение между локалното състояние и Binance. Findings описват откритите несъответствия, severity, текущ resolution status и евентуално извършено healing действие. Това е по-надеждно от обикновен лог, защото unresolved finding остава видим и може да участва в последващ operator workflow.

Operational данните допълват trading историята. Service heartbeat показва жизнеността на приложение, alert регистрира значим проблем, а audit записва човешката реакция. Така техническият проблем и операторското действие не се смесват в един неструктуриран log stream.

Фигура 5.1. Логическо разделение на основните persistence области



5.6. Performance analytics и IPerformanceAnalyticsStore

IPerformanceAnalyticsStore отделя изчислителните модули от конкретната SQL структура. Backtest, optimization и други research процеси подават структурирани резултати към store, който ги съхранява в performance_runs, performance_snapshots, performance_trades, optimization_trials и walk_forward_windows.

performance_runs идентифицира конкретно аналитично изпълнение чрез bot, strategy version, symbol, interval, период и run type. Snapshot-ите пазят обобщени показатели като net profit, win rate, maximum drawdown и score, а performance_trades позволяват анализ на ниво отделна сделка.

Това позволява резултатите да бъдат сравнявани между версии на стратегията, без старите стойности да се презаписват. Всеки нов експеримент създава отделен run, което превръща analytics слоя и в история на направените изследвания.

5.7. Optimization процес и parameter search

Optimization процесът започва с дефиниране на ограничено parameter space. За TP-only Grid семейството това са естествено PriceDistance, ProfitDistance, OrderSideLimit и CooldownSeconds. Всеки кандидат се оценява върху еднакъв исторически набор и еднакви execution assumptions, за да бъде сравнението коректно.

За всеки trial се съхраняват параметрите, score и основните performance показатели. Score не трябва да се интерпретира като гаранция за печалба. Неговата цел е да даде общ критерий за сортиране, който може да съчетава доходност, drawdown, win rate и брой приключени позиции.

При BOT8012 optimization е особено подходящ, защото стратегията е параметризирана и има ясно измерими trade-offs. По-малък PriceDistance може да увеличи честотата на entries, но и да повиши експозицията. По-голям ProfitDistance увеличава целта на сделката, но може да намали вероятността за достигане на take-profit.

5.8. Walk-forward анализ и ограничаване на overfitting

Резултатът от optimization върху един исторически период може да бъде подвеждащ, ако параметрите са прекалено пригодени към конкретните движения в него.[7][8] Walk-forward подходът разделя данните на последователни train и test прозорци. Параметрите се избират върху in-sample сегмента и след това се оценяват върху unseen out-of-sample сегмент.

Следващият прозорец се измества напред и процесът се повтаря. Ако една конфигурация работи добре само върху training периода, но се влошава рязко върху test сегментите, това е сигнал за нестабилност. Затова walk-forward резултатът е по-информативен от единичен най-добър optimization trial.

Системата съхранява прозорците отделно, което позволява Dashboard да показва не само общ score, а и поведението на стратегията във времето. Това е важно при криптопазар, където volatility режимите могат да се различават значително.

5.9. Backtesting и optimization като versioned run-ове

Backtest и optimization се пазят като versioned run-ове. Всеки run е свързан с конкретен bot, strategy version и configuration snapshot. Така резултатът остава възпроизводим и след бъдеща промяна на runtime параметрите. При сравнение между два run-а трябва да се отчитат не само net profit и win rate, а и maximum drawdown, брой сделки, средна продължителност и използваните fees/slippage

assumptions. Това позволява да се разграничи по-висока доходност, постигната с прекомерен риск, от по-стабилно поведение.

5.10. Jobs Worker и трайният lifecycle на дългите операции

Historical ingestion, backtesting, optimization и replay могат да работят значително по-дълго от нормална HTTP заявка. Dashboard API затова не извършва цялата операция синхронно, а записва job. Jobs Worker claim-ва задачата, изпълнява я и периодически обновява progress и status.

Job записът съдържа attempt count, processing worker, request payload, next attempt time, error и completion metadata. При transient failure задачата може да бъде върната в Pending, докато permanent failure трябва да приключи без безкрайно повторение. Това прави recovery поведението видимо и тестируемо. Отделянето на long-running job от HTTP request е важно и за operator experience. Web интерфейсът получава бърз Accepted отговор, а след това следи progression чрез polling. Ако браузърът бъде затворен, задачата продължава независимо.

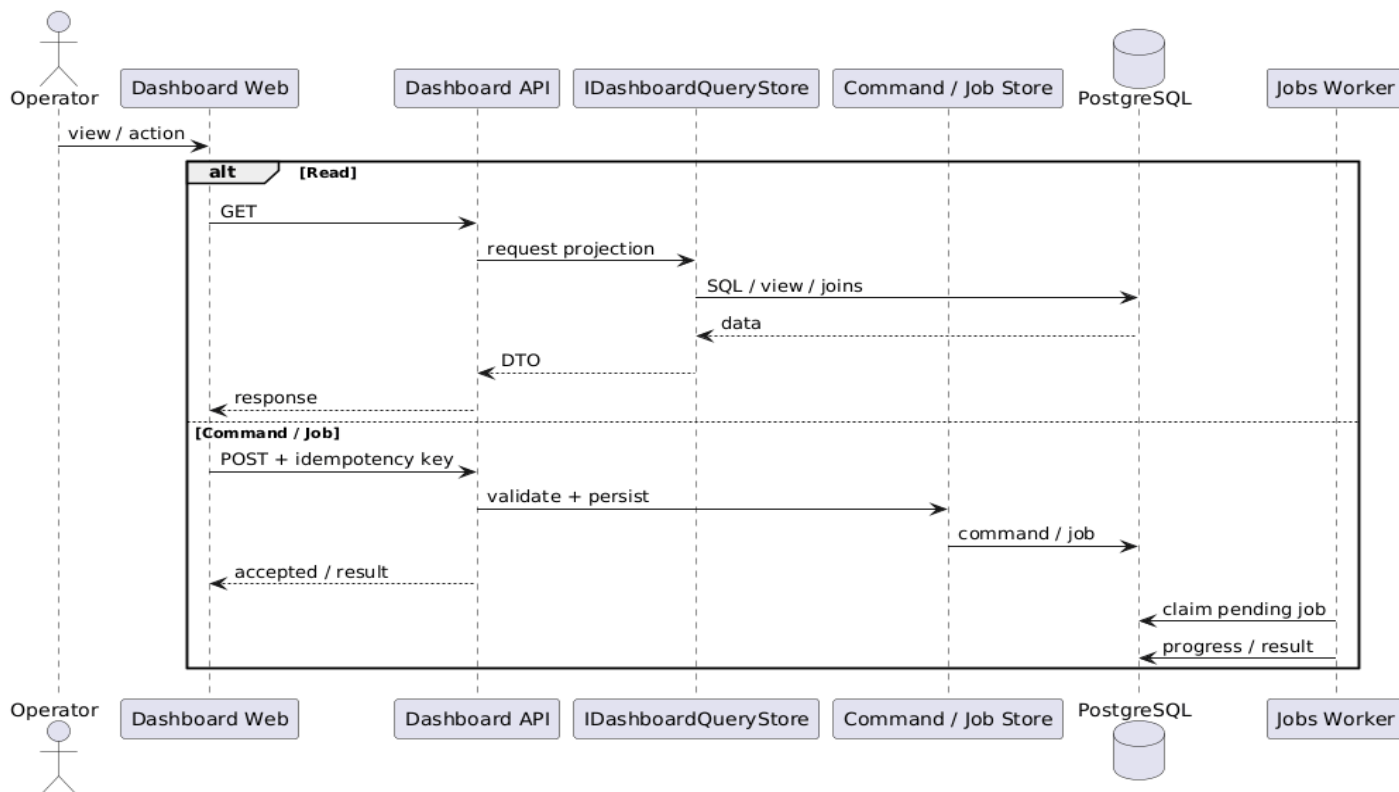
5.11. Dashboard Application слой и query projections

IDashboardQueryStore отделя API contract-а от конкретните таблици и joins. Controller-ът заявява определена projection, а persistence реализацията решава дали тя идва от една таблица, view или комбинация от schemas. Това позволява SQL да бъде оптимизиран независимо от frontend.

Overview е пример за агрегирана projection. Той комбинира текущ PnL, open positions, critical alerts и bot runtime snapshots. Signals и Positions страниците използват по-детайлни read модели, докато Analytics и Optimization четат performance таблиците.

Write операции не се изпълняват през query store. Bot command, configuration update, alert acknowledge и enqueue на job имат отделни application contracts. Това разделя четенето на състояние от действията, които създават side effect.

Фигура 5.3. Dashboard query, command и job boundaries

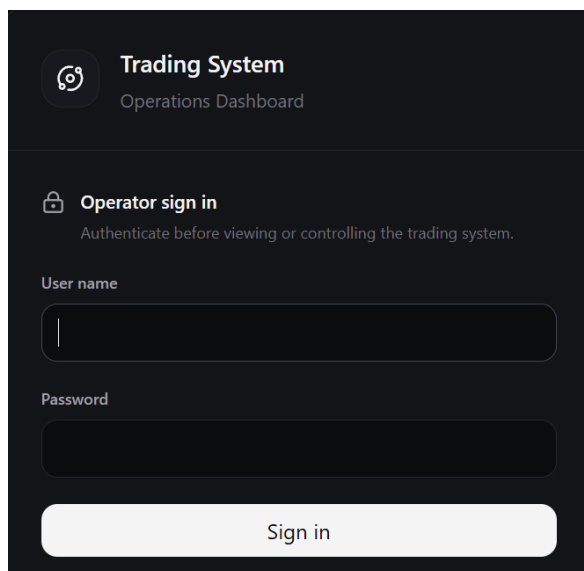


5.12. Dashboard Web като визуализация на projections

Dashboard Web използва React и TypeScript и не е authoritative източник на търговско състояние. Страници като Overview, Bots, Signals, Positions, Paper Trading, Backtests, Optimization, Replays, Health, Alerts и Audit визуализират данни, получени от API.

При write операция frontend изпраща command или job request и след това обновява съответната projection. React Query управлява polling и invalidation. Това е особено полезно при optimization или replay, където резултатът се появява след приключване на background job, а не в първоначалния HTTP response.

Фигура 5.4. Екран за автентикация на оператор в уеб таблото за управление



5.13. Runtime configuration, versioning и audit reason

Runtime configuration позволява параметри като environment, quantity, leverage, signal mode, PriceDistance, ProfitDistance, side limits и cooldown да бъдат променяни. Всяка промяна трябва да съдържа reason, защото тя е операторско действие и следва да остане в audit trail.

Конфигурацията е versioned. При update се проверява текущата версия, за да се избегне загуба на паралелна промяна. Configuration refresh worker периодично чете новите версии и обновява in-memory provider. Така hot path не зависи от SQL заявка при всеки signal.

Важно е да се разграничават configuration state и runtime state. „Configured“ означава, че ботът има валидна конфигурация. „Running“ означава, че execution е активиран. Бот може да бъде Configured, но Stopped, или Running с конкретна versioned configuration.

5.14. Операторски команди и command history

Runtime управлението се извършва чрез команди като Start, Pause, Resume, Stop и EmergencyStop. Командата се записва трайно с requested by, reason, timestamp и status. StrategyService обработва заявката и променя runtime state отделно от strategy configuration.

Command history има две роли. Първо, операторът вижда дали действието е прието и завършено. Второ, при troubleshooting може да се установи дали промяна в поведението е причинена от автоматична логика или от човешка команда. По този начин control plane не е просто набор от бутони, а аудитируем operational интерфейс.

За по-рискови действия са необходими допълнителни confirmation механизми. Това е особено важно за EmergencyStop, cancel на поръчки или затваряне на отворени позиции, защото те могат да променят exchange state.

5.15. Authentication, authorization и API hardening

Операторският Dashboard е защитен чрез authentication и role-based authorization. Read endpoints могат да бъдат достъпни за viewer роля, докато configuration updates, bot commands и job creation изискват Operator или Administrator права. Това намалява риска интерфейсът за наблюдение да се превърне неволно в execution канал.

Паролата не се съхранява като plaintext, а като PBKDF2-SHA256 hash. След успешно вписване backend издава token с ограничен lifetime. API write операциите използват validation и rate limiting, а определени endpoints поддържат idempotency key, така че повторен browser request да не създаде втори side effect.

Correlation identifier се използва за свързване на HTTP request с логовете и последващите операции. Това улеснява диагностика при 500 response или distributed failure, защото една и съща заявка може да бъде проследена през няколко компонента.

5.16. Observability данни и service health

Healthcheck и heartbeat отговарят на различни въпроси. Healthcheck показва дали конкретен process или endpoint в момента е достъпен. Heartbeat показва кога приложението последно е изпълнило свой operational lifecycle и е записало жизненост. Затова container може да е стартиран, но stale heartbeat да покаже проблем във вътрешната работа.

Dashboard съхранява service heartbeats трайно и показва компонентите като Healthy или Unhealthy според последното наблюдение. Alerts допълват тази картина с persistent проблеми, а audit показва операторски acknowledge или други действия.

Prometheus-compatible metrics са количественият слой на observability. Те позволяват да се измерват request duration, errors, memory, processing times и други показатели, които не могат да бъдат представени само чрез бинарен health status.

5.17. Database migrations и надеждност на deployment-a

Тъй като persistence моделът се развива заедно със системата, schema changes се прилагат чрез последователни SQL migrations. Те създават и разширяват trading history, dashboard backend, runtime orchestration, jobs, service health, Paper Trading, plugin system, replay и Event Store функционалности.

Migration tracking предотвратява неявно повторно прилагане на една и съща промяна и позволява checksum проверка. Това е важно при deployment върху нова среда или след възстановяване на празен volume. Без migrations API може да стартира като process, но read projections да връщат грешки за липсващи views или tables.

Практическата локална верификация показва именно тази разлика: health endpoint може да бъде успешен, докато Overview заявка се проваля поради липсваща database schema. Затова инфраструктурният healthcheck не трябва да се приема като доказателство, че application data layer е напълно готов.

5.18. Deployment върху Docker Compose и Synology NAS

Основните услуги се стартират чрез Docker Compose. PostgreSQL и Redis използват persistent volumes, а application services комуникират през вътрешна Docker network. Към host се expose-ват само необходимите входни точки, като Dashboard Web, webhook endpoint и metrics. Dashboard Web се обслужва от Nginx, който проху-ва /api/ към Dashboard API и скрива вътрешната service topology. Synology NAS се използва като демонстрационна deployment среда, в която могат да се проверят container health, restart behavior, persistence и достъпът до Dashboard от отделна машина.

ГЛАВА 6. ТЕСТВАНЕ И ПРАКТИЧЕСКА ВЕРИФИКАЦИЯ

6.1. Подход към практическата верификация

Практическата верификация на разработената система е организирана като последователност от контролирани сценарии, чрез които се проверява поведението на основните компоненти в реално работеща контейнеризирана среда. Целта не е да се провери единствено дали даден алгоритъм може да генерира LONG или SHORT решение, а дали цялата последователност от получаване на сигнал до изпълнение, съхраняване и визуализация работи съгласувано.

Тестовата среда включва PostgreSQL, Redis, MarketDataService, IndicatorServices, StrategyService, UserStreamService, TradingJobsWorker, Dashboard API, Dashboard Web и TradingView webhook услугата. По този начин могат едновременно да се наблюдават runtime състоянието на услугите, получените сигнали, решенията на TradingEngine, persistence записите, Binance Demo account state и операторските известия.

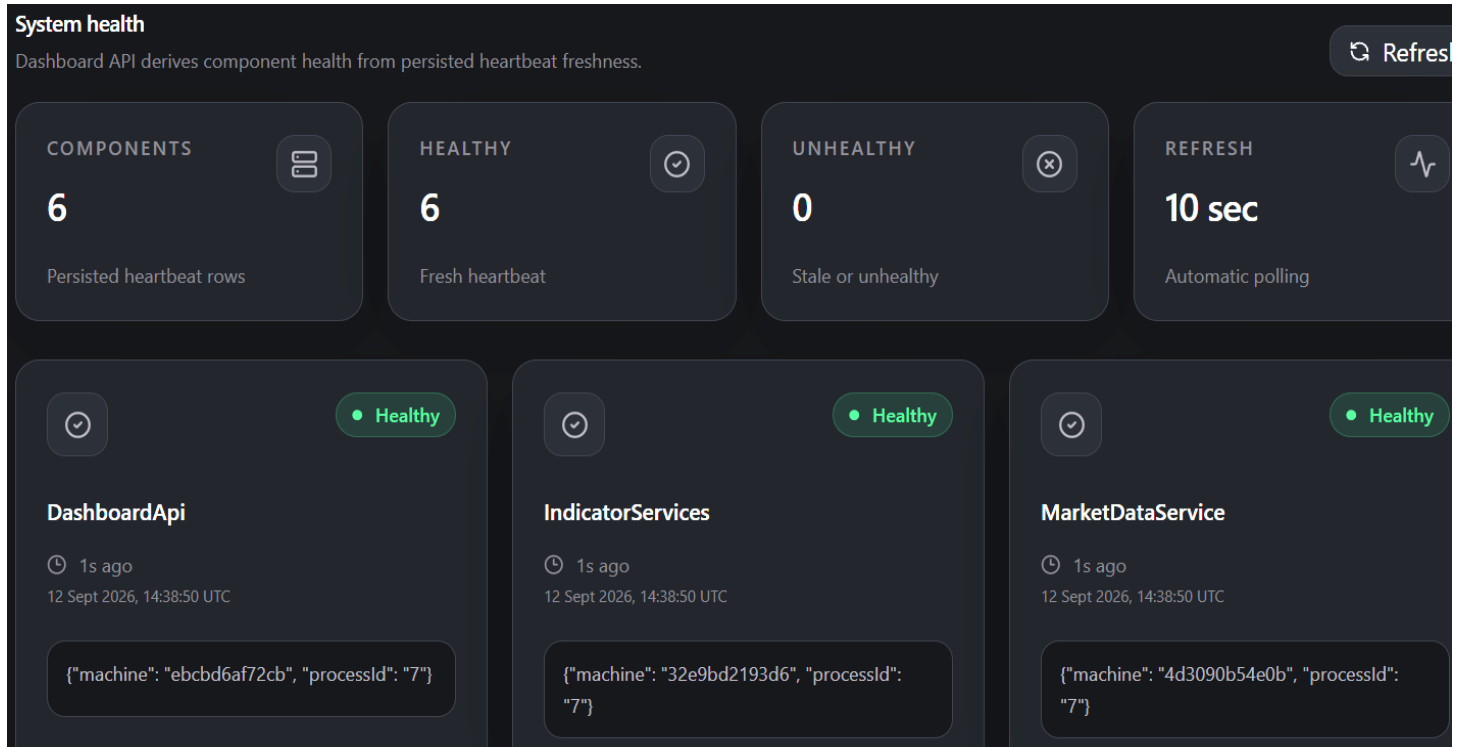
За основен демонстрационен сценарий е използван BOT8012 с TP-only Grid стратегия. Този бот е подходящ за практическата проверка, защото позволява едновременно да бъдат демонстрирани външен TradingView сигнал, runtime конфигурация, risk и strategy processing, Binance Demo execution, take-profit поръчка, Dashboard projections и аналитични модули. Отделно е използван BOT8013 в Paper среда. Чрез него се проверява, че същият application pipeline може да завърши със симулирано изпълнение, без да създава външна Binance поръчка.

6.2. Проверка на стартирането и инфраструктурната готовност

Първата стъпка при практическата проверка е установяване дали основните услуги са стартирали успешно и дали могат да комуникират с необходимите инфраструктурни зависимости.

Системата се стартира чрез Docker Compose. След стартиране се проверяват състоянието на контейнерите, application heartbeats и Dashboard health projection. Важно е да се прави разграничение между работещ процес и действителна готовност на application слоя. Например контейнерът може да отговаря на health endpoint, но read заявките да не могат да се изпълнят, ако PostgreSQL схемата не е приложена коректно. Поради това database migrations са част от startup verification процеса. След тях Dashboard API трябва успешно да зарежда projections като Bots, Signals, Positions и останалите operational данни. При наблюдавания сценарий основните приложения DashboardApi, IndicatorServices, MarketDataService, StrategyService, TradingJobsWorker и UserStreamService се визуализират като Healthy. Това показва, че системата е достигнала състояние, подходящо за следващите практически сценарии.

Фигура 6.1. Състояние на основните runtime компоненти в операторския Dashboard



6.3. Проверка на runtime управлението и конфигурацията на BOT8012

След установяване на инфраструктурната готовност се проверява runtime управлението на търговските ботове. Runtime състоянието и конфигурацията са разделени. Наличието на валидна конфигурация не означава автоматично, че ботът може да изпълнява операции. Чрез команди като Start, Pause, Resume и Stop се управлява дали нови сигнали могат да достигнат до execution слой.

За демонстрацията BOT8012 е стартиран чрез операторския Dashboard. След успешно изпълнение на командата runtime status преминава в Running. Тази операция е трайно проследима. Command и audit записите съдържат потребител, момент на изпълнение, reason и резултат, което позволява по-късно да бъде установено дали определена промяна в поведението е предизвикана от стратегията или от оператор.

Фигура 6.2. Runtime състояние на BOT8012 в операторския Dashboard

BOT	STATUS	ENVIRONMENT	LAST SIGNAL	LAST DECISION
BOT8012 v1.0.0	● Running	Demo	Long 17h ago	Open Position opened

Освен runtime status се проверява и конкретната конфигурация, използвана при демонстрационния сценарий. BOT8012 е конфигуриран със следните основни параметри: Quantity = 0.002, Leverage = 50, CooldownSeconds = 180, PriceDistance = 400, ProfitDistance = 200, OrderSideLimit = 2, разрешени LONG и SHORT позиции. Тези стойности са особено подходящи за проверка, защото част от тях могат впоследствие да бъдат сравнени с действително създадената Binance Demo позиция.

Фигура 6.3. Основни runtime и стратегически параметри на BOT8012

Execution

Quantity, leverage and permitted trade directions.

Quantity

Leverage

Cooldown (seconds)

0.002

50

180

Enable Long

Allow the strategy to open long positions.

☒

Enable Short

Allow the strategy to open short positions.

☒

Strategy parameters

Nullable parameters remain empty when they are not used by the selected strategy.

Price distance

Profit distance

Order side limit

400

200

2

6.4. Проверка на TradingView сигнал и TradingEngine pipeline

BOT8012 работи в режим TradingViewOnly. Това означава, че външният сигнал се получава през TradingView webhook, но webhook компонентът не изпраща директно поръчка към Binance. Полученият payload първо се валидира и нормализира към вътрешния TradeSignal модел. След това сигналът се публикува към общия processing pipeline.

За практическата проверка е подаден контролиран LONG сигнал за BTCUSDC с уникален SignalId. След получаването му TradingEngine извършва последователните проверки за timestamp, идемпотентност, runtime състояние, operation lock, cooldown, bot-specific strategy policy и централизирания risk layer. Едва когато всички необходими проверки разрешат операцията, execution слойът може да бъде извикан. Резултатът е видим в Signals страницата на Dashboard. Записът показва BOT8012, BTCUSDC, LONG, източник TRADINGVIEW, среда Demo и решение Open с причина Position opened. Този резултат е важен, защото показва, че външният сигнал не е заобиколил application safety слоя, а е преминал през нормалния TradingEngine processing flow.

Фигура 6.4. Успешно обработен TradingView сигнал за BOT8012

11 Sept 2026, 21:24:30	BOT8012 1.0.0	BTCUSDC	● Long	—	TRADINGVIEW Demo	● Open Position opened
11 Sept 2026, 21:19:59	BOT8013 1.0.0	BTCUSDC	● LONG	77,350	internal-indicators Paper	● Pending

6.5. Binance Demo end-to-end сценарий с BOT8012

След успешната обработка на сигнала е извършена пълна end-to-end проверка в Binance Demo. При приетия LONG сигнал BOT8012 използва конфигурирано количество 0.002 BTC. Системата

създава Market операция за отваряне на позицията и след нейното потвърждение създава take-profit поръчка според конфигурацията на TP-only Grid стратегията.

В наблюдавания сценарий е отворена BTCUSDC LONG позиция с: quantity 0.002, entry price 77 359.9, take-profit 77 559.9. Разликата между entry и take-profit е точно 200, което съответства на конфигурирания ProfitDistance = 200. Dashboard Positions визуализира същата позиция като Open и показва position identifier, side, quantity, entry и take-profit.

Фигура 6.5. Отворена Демо позиция на BOT8012 в операторския Dashboard

Position history				
Filter by bot, symbol or persisted lifecycle status.				
SIDE	STATUS	QUANTITY	ENTRY	TAKE PROFIT
● Long	● Open	0.002	77,359.9	77,559.9

Binance Demo order history предоставя независимо потвърждение на изпълнението. Entry поръчката е Market операция от тип Open Long и е изпълнена за количество 0.002 BTC при средна цена 77 359.9.

Фигура 6.6. Изпълнена Market поръчка за отваряне на BTCUSDC LONG позиция в Binance Demo

Symbol	Order Type	Side	Price	Avg Price	Executed	Amount	Trigger Conditions	Status
BTCUSDC Perpetual	Market	Open Long	Market	77,359.9	0.002 BTC	0.002 BTC	-	Filled

Фигура 6.7. Създадена Take-Profit поръчка на BOT8012 в Binance Demo

Time	Symbol	Order Type	Side	Price	Filled	Order Amount
2026-09-12 00:24:33	BTCUSDC Perpetual	Limit	Close Long	77,559.9	0.000 BTC	0.002 BTC

Така може директно да бъде свързана runtime конфигурацията ProfitDistance = 200 с действително генерираната борсова поръчка. Успешното изпълнение се потвърждава и чрез Telegram notification. Известиято съдържа bot, symbol, side, position identifier и source.



6.6. UserStream и последващ lifecycle на борсовата операция

REST заявката към Binance не представлява целия жизнен цикъл на една търговска операция. След създаването на поръчката Binance може да изпраща допълнителни private events чрез UserStream.

Поръчката може да премине през различни състояния, например NEW, PARTIALLY_FILLED, FILLED или CANCELED. UserStreamService получава тези външни събития, нормализира ги и ги предава към съответните lifecycle handlers. Това е важно, защото локалното състояние не трябва да се базира единствено върху това каква заявка системата е изпратила към борсата. То трябва да отразява потвърденото от Binance състояние. При TP-only Grid сценария особено важно е състоянието на entry и take-profit поръчките. Създадените exchange събития участват в последващото обновяване на trading history и локалните projections.

Операторско затваряне чрез ClosePosition следва същия двупосочен модел: локално се заявява действие, а действителният exchange lifecycle трябва впоследствие да бъде потвърден чрез Binance state и съответните order events.

6.7. Persistence, Dashboard projections и audit trail

След execution се проверява дали едно и също събитие е представено последователно в различните persistence и read модели. Signals съдържа входния TradeSignal и резултата от processing-a. Positions съдържа текущото състояние на позицията. Други projections използват същите трайни данни за агрегирана визуализация и анализ.

Dashboard не е authoritative state сам по себе си. Той представлява визуализация на данни от persistence и application query слой. Поради това коректността се оценява чрез съгласуваност между exchange state, локалните записи и визуализираните projections.

Отделно audit механизмът позволява операторските действия да бъдат проследени. При стартирането на BOT8012 е записано събитие BotCommand.Start.Completed с потребител, момент на изпълнение и reason.

Фигура 6.9. Audit следа при изпълнение на операторска команда за BOT8012

BotCommand.Start.Completed
BotCommand

Yoanka
11 Sept 2026, 21:13:19 UTC

ENTITY ID	REASON
40ba1bb2-5a99-4f65-8a53-3f8b84e1b58f	Test bot 12
CORRELATION ID	IP ADDRESS
📄 40ba1bb2-5a99-4f65-8a53-3f8b84e1b58f	—

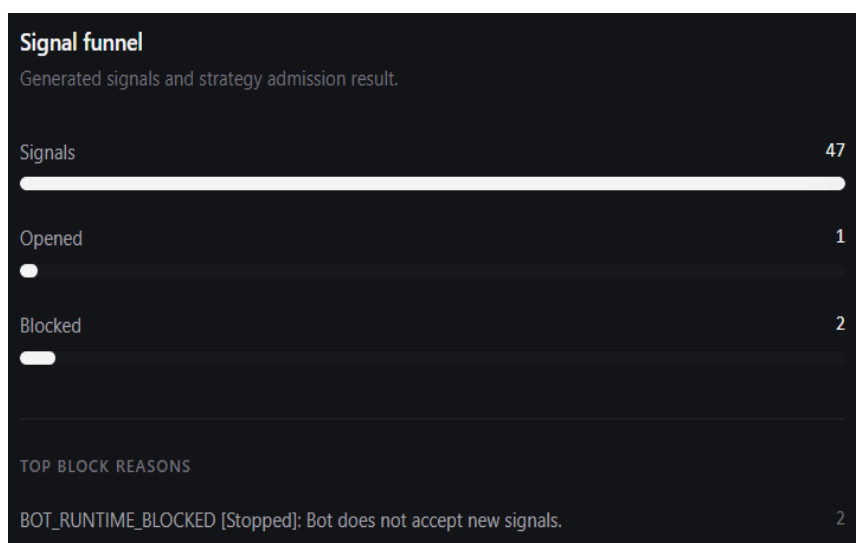
POST /api/v1/bots/BOT8012/commands
HttpRequest

Yoanka
11 Sept 2026, 21:13:17 UTC

ENTITY ID	REASON
BOT8012	—
CORRELATION ID	IP ADDRESS
📄 78016d29-3c61-4886-8a24-495225e06ca2	::ffff:172.18.0.7

Dashboard предоставя и агрегирана визуализация на обработените сигнали и причините за admission или блокиране.

Фигура 6.10. Обобщение на обработените сигнали и резултатите от strategy admission



6.8. Проверка на Paper Trading режима

Отделен practically-oriented сценарий е изпълнен за Paper Trading режима. За тази проверка BOT8013 е конфигуриран с execution environment Paper. След промяната на execution environment StrategyService е рестартиран, тъй като смяната между Paper, Demo и Production изисква runtime restart преди новата среда да стане активна. След активиране на BOT8013 е подаден контролиран LONG сигнал за BTCUSDC. Сигналят преминава през нормалния TradingEngine pipeline и общите проверки за runtime state, strategy policy и risk. Разликата спрямо Demo сценария е само в execution boundary. Вместо реална Binance Demo операция се използва Paper executor.

В резултат е създадена симулирана LONG позиция със следните параметри: bot: BOT8013; symbol: BTCUSDC; side: LONG; quantity: 0.002; entry price: 77 344.1336; take-profit: 77 544.1336; stop-loss: 76 764.0526; entry fee: 0.061875. Позицията е видима в Paper Trading Dashboard като Open position.

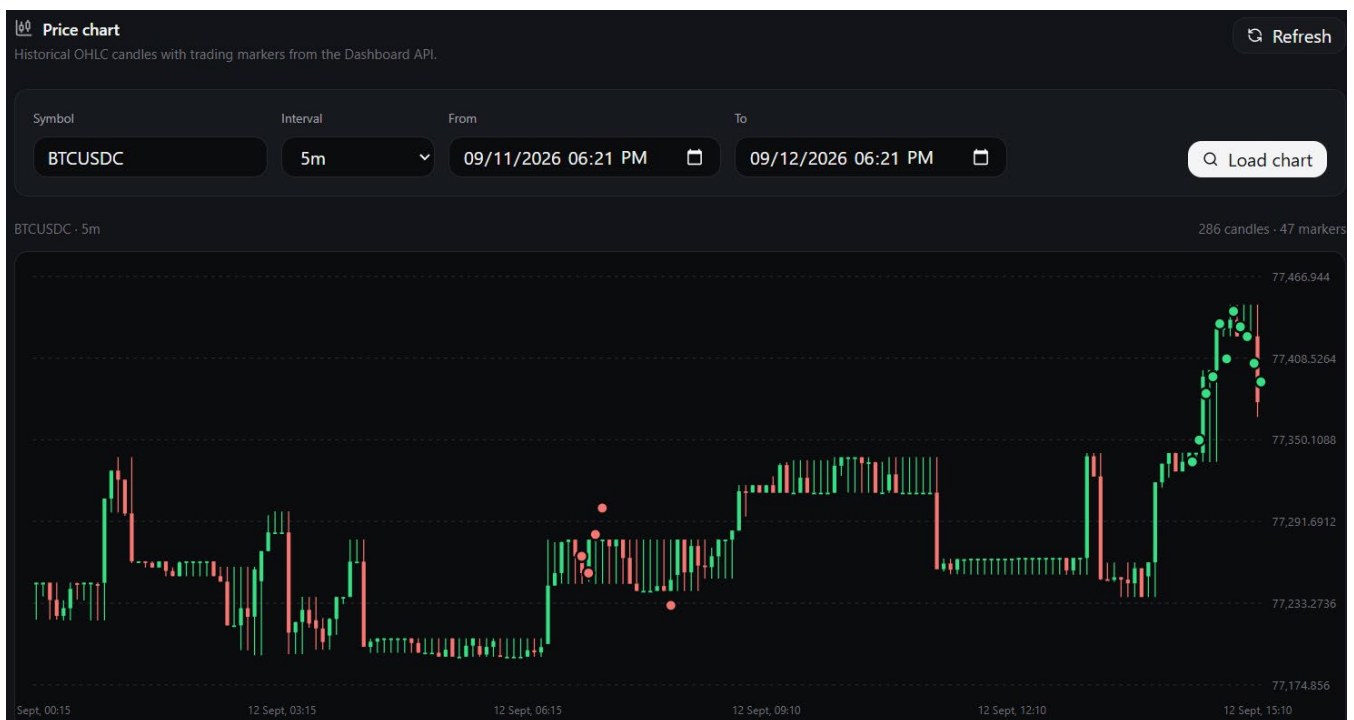
Фигура 6.11. Отворена симулирана Paper Trading позиция за BOT8013

POSITION	SYMBOL	SIDE	QUANTITY	ENTRY	TAKE PROFIT	STOP LOSS	ENTRY FEE
P2609121701324559 BOT8013	BTCUSDC	Long	0.002	77,344.1336	77,544.1336	76,764.0526	0.061875

6.9. Проверка на историческите пазарни данни

Historical data модулът позволява съхраняваните пазарни данни да бъдат използвани независимо от текущия live stream. За визуалната проверка е зареден BTCUSDC chart с 5-минутни свеци. Dashboard визуализира историческите OHLC данни заедно с persisted trading markers. Тази визуализация е полезна като връзка между market history и последващите analytical операции. Маркерите позволяват trading събитията да бъдат разглеждани в контекста на движението на цената, вместо единствено като редове в база данни.

Фигура 6.12. Исторически BTCUSDC market chart с persisted trading markers

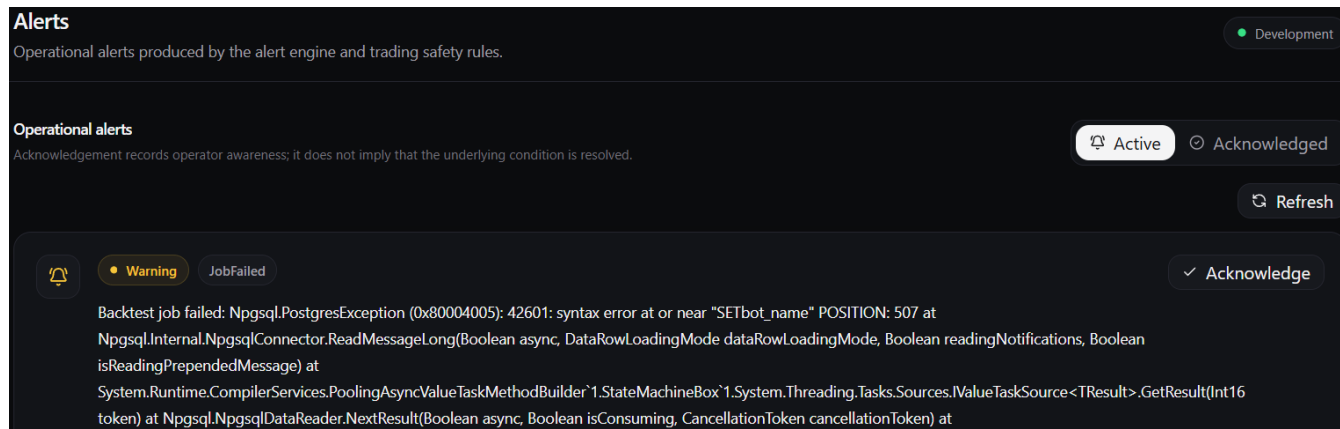


6.10. Проверка на operational alerts и failure visibility

Практическата верификация включва не само успешните сценарии, но и начина, по който системата прави проблемите видими за оператора. При първоначален backtest run е възникнал SQL

проблем, в резултат на който background job е приключил неуспешно. Системата е създавала operational alert от тип JobFailed, който се визуализира в Dashboard. Неуспешната background операция не остава единствено в application logs, а се превръща в трайно operational събитие, което операторът може да види и acknowledge-не. След отстраняване на причината backtest процесът е изпълнен повторно успешно.

Фигура 6.13. Operational alert при неуспешно изпълнение на backtest job



6.11. Backtesting на BOT8012

След подготовка на историческия вход са изпълнени няколко backtest run-a за BOT8012.

Понеже BOT8012 работи в TradingViewOnly режим, backtest входът включва исторически candles и контролиран набор от external-style сигнали. Част от използвания набор е синтетично създаден специално за теста и е маркиран като такъв. Той не трябва да се интерпретира като реална архивирана TradingView история. Целта е различни конфигурации да бъдат оценявани върху еднакъв исторически диапазон и еднакъв входен signal dataset. Сравнени са две конфигурации:

- вариант с PriceDistance 300 и ProfitDistance 150;
- базова конфигурация с PriceDistance 400 и ProfitDistance 200.

При наблюдавания тест първата конфигурация достига net profit 4.84 и win rate 94.74%, докато базовата конфигурация достига net profit 3.23 и win rate 95.00%. Maximum drawdown е съответно приблизително 0.04% и 0.03%.

Фигура 6.14. Сравнение на два backtest run-a на BOT8012 с различни стратегически параметри

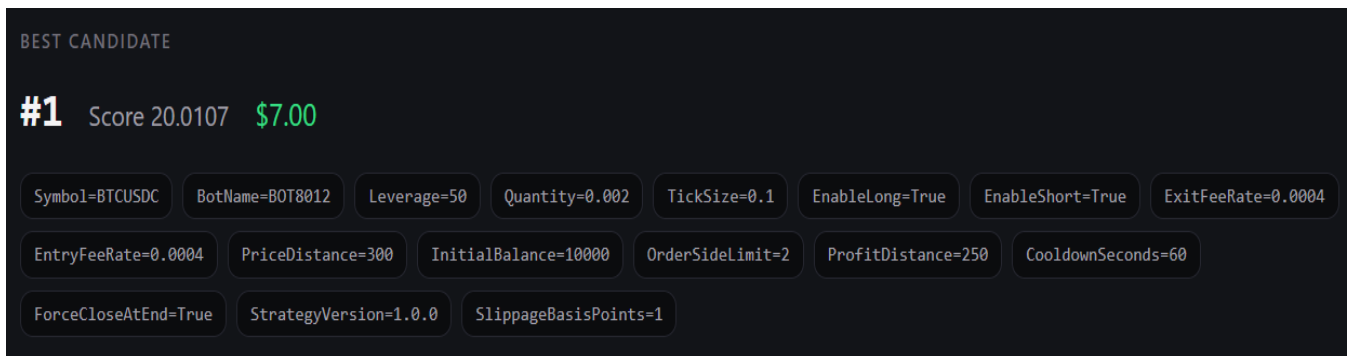
RUN	BOT	MARKET	STATUS	NET PROFIT	WIN RATE	MAX DD
b91a4a78-f94a-4157-a83f-cc3... Backtest	BOT8012 v1.0.0	BTCUSDC 1m	Completed	\$4.84	94.74%	0.04%
24222a7c-ffe0-499b-97d1-f95... Backtest	BOT8012 v1.0.0	BTCUSDC 1m	Completed	\$3.23	95.00%	0.03%

6.12. Optimization на стратегическите параметри

След backtesting е проверен и optimization модулът. За BOT8012 е дефинирано ограничено parameter space върху: PriceDistance, ProfitDistance, OrderSideLimit, CooldownSeconds. В наблюдавания optimization run са оценени множество комбинации върху еднакъв historical dataset. Резултатите се записват като отделни trials и се подреждат според score функцията, която комбинира performance и risk показатели.

Най-високо класираният candidate при конкретния използван набор от данни съдържа: PriceDistance = 300, ProfitDistance = 250, OrderSideLimit = 2, CooldownSeconds = 60. Полученият score е 20.0107, net profit е 7.00, win rate е 95.00%, maximum drawdown е приблизително 0.04%, а броят на приключените позиции е 20.

Фигура 6.15. Ranked optimization резултати и избрана конфигурация за BOT8012

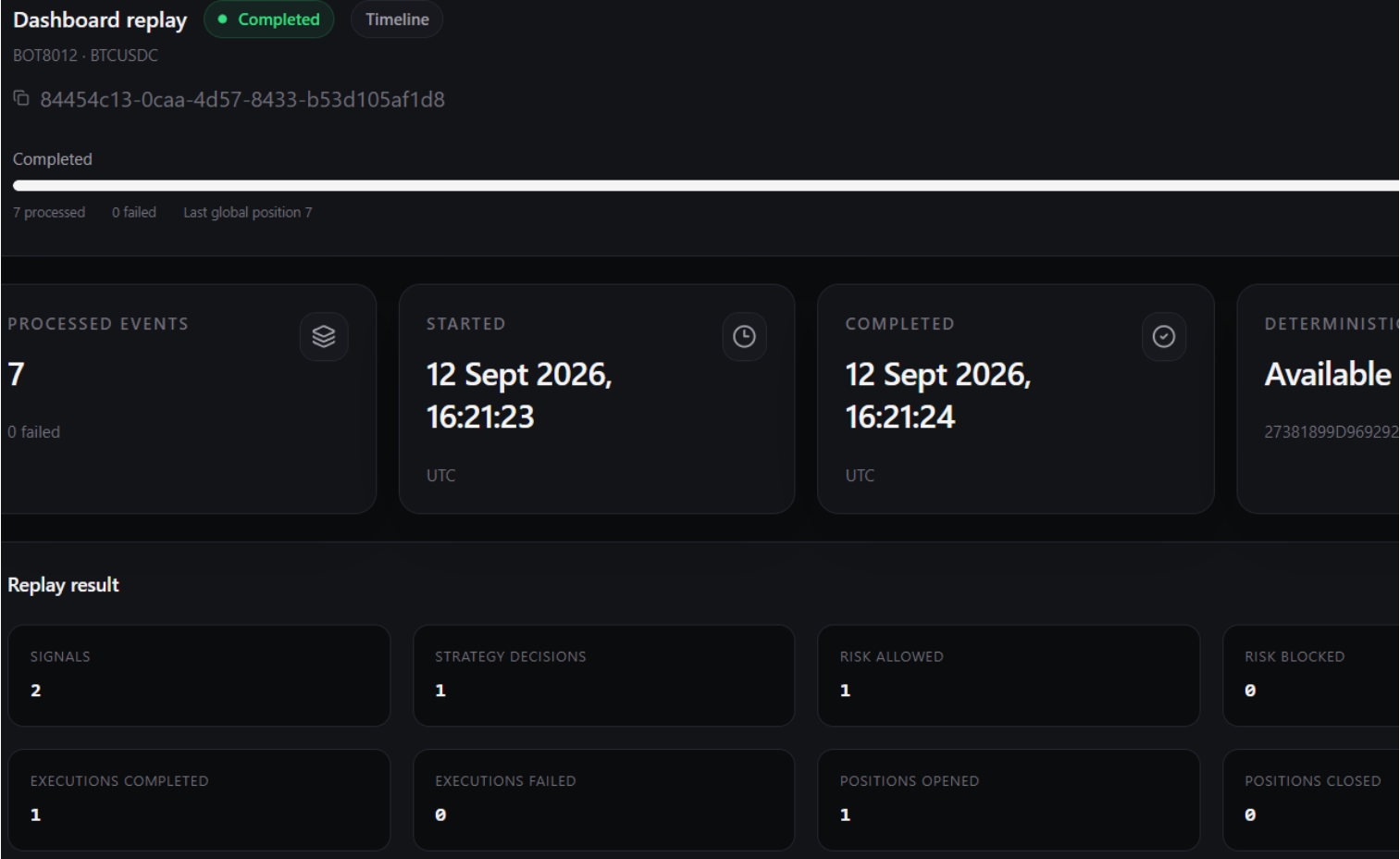


Този резултат не трябва да се интерпретира като универсално най-добра конфигурация. Той е най-добрият кандидат за конкретния historical dataset, избрания период, parameter space и използваната score функция.

6.13. Replay и детерминистично възпроизвеждане

Replay механизмът е проверен чрез отделен background run за BOT8012. За разлика от backtesting, replay не цели да симулира множество алтернативни параметри. Неговата задача е да възпроизведе вече persist-ната последователност от исторически събития. При изпълнения тест replay job приключва със статус Completed. Persisted replay steps показват последователно: trading.signal.received→trading.strategy.decision-taken→trading.risk.decision-taken→trading.execution.requested→trading.execution.completed→trading.position.opened.

Фигура 6.16. Успешно детерминистично replay изпълнение на исторически събития за BOT8012



6.14. Проверка на идемпотентност, cooldown и risk ограничения

Надеждността на TradingEngine зависи и от сценарии, при които сигналът не трябва да достигне до execution. При повторен SignalId логическата идемпотентност трябва да предотврати втори lifecycle за същия сигнал.

Operation lock решава различен проблем. Той сериализира конфликтни операции за една и съща комбинация bot, symbol и side и ограничава възможността два паралелни сигнала едновременно да променят едно и също trading state.

Cooldown ограничава прекалено бързото последващо execution действие след вече приета операция.

CentralRiskManager поставя още една независима граница. Сигналът може да е валиден и strategy policy да позволява entry, но при недопустима projected exposure или друго критично portfolio състояние execution трябва да бъде блокирано.

По този начин TradeSignal представлява само намерение за действие, а не директна борсова поръчка.

6.15. Reconciliation и възстановяване след промяна на runtime състоянието

Разпределената архитектура не позволява PostgreSQL, Redis и Binance да участват в една обща атомарна транзакция. Поради това след restart, мрежов проблем, пропуснат UserStream event или промяна в exchange state е възможно временно разминаване между локалното и външното състояние. Reconciliation слоят съществува, за да сравнява тези две гледни точки и да създава findings при открити несъответствия.

В практическия процес е наблюдавано и fail-closed поведение, при което наличен critical reconciliation finding може да блокира нов сигнал, вместо системата да продължи автоматично при неясно състояние. Това е важна safety характеристика: неизвестното състояние не се интерпретира като разрешение за нова експозиция.

Не всички възможни infrastructure failure сценарии са изпълнявани като отделен chaos test. Поради това resilience механизмите се разглеждат тук основно чрез реализираното restart, persistence и reconciliation поведение, а не като твърдение за пълно fault-injection тестване.

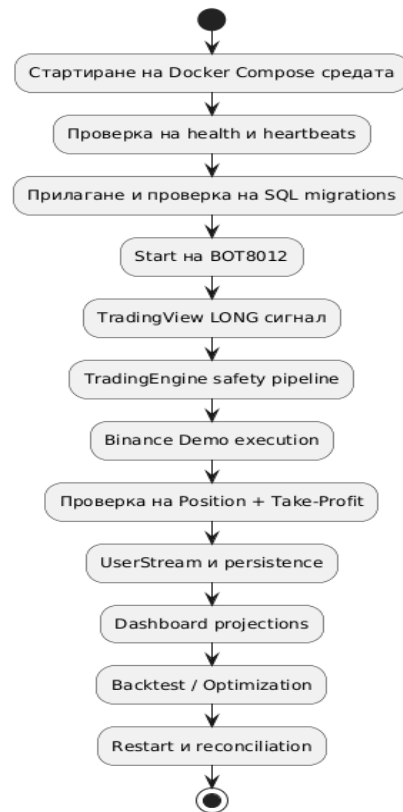
6.16. Обобщение на проверените практически сценарии

Сценарий	Проверяван механизъм	Наблюдаван резултат
Стартиране на Compose средата	Infrastructure readiness	Основните runtime услуги са Healthy и подават heartbeat
Runtime Start на BOT8012	Runtime orchestration	BOT8012 преминава в Running
Runtime configuration	Versioned bot configuration	Dashboard показва quantity, leverage, cooldown и strategy parameters
TradingView LONG за BOT8012	Signal processing	Сигналът достига TradingEngine и приключва с Open

Binance Demo LONG	Execution	Отворена е BTCUSDC LONG позиция с quantity 0.002
Take-Profit	Bot-specific execution	Създадена е Limit Close Long поръчка на entry + 200
Telegram notification	Operational notification	Получено е известие за успешно отворената позиция
Audit	Operator traceability	Start командата и HTTP действието са записани
Paper LONG за BOT8013	Simulated execution	Създадена е отделна Paper позиция без Binance order
Historical chart	Historical data	Candle history и persisted markers се визуализират
Failed backtest	Alerting	Създаден е persistent JobFailed alert
Backtest comparison	Research pipeline	Два parameter set-a са сравнени чрез performance metrics
Optimization	Parameter search	Trials са ranked и е избран best candidate
Replay	Deterministic event processing	7 събития са обработени успешно и са налични replay steps
Risk/reconciliation block	Fail-closed safety	При критично несъответствие ново execution действие може да бъде блокирано

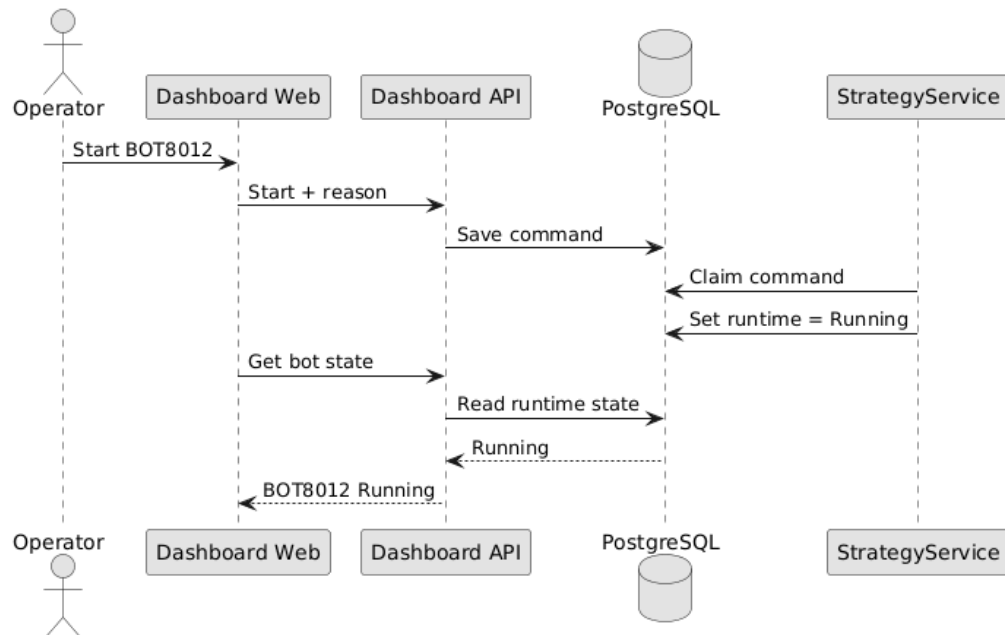
6.17. Диаграма на практическата верификация

Нива на практическа верификация

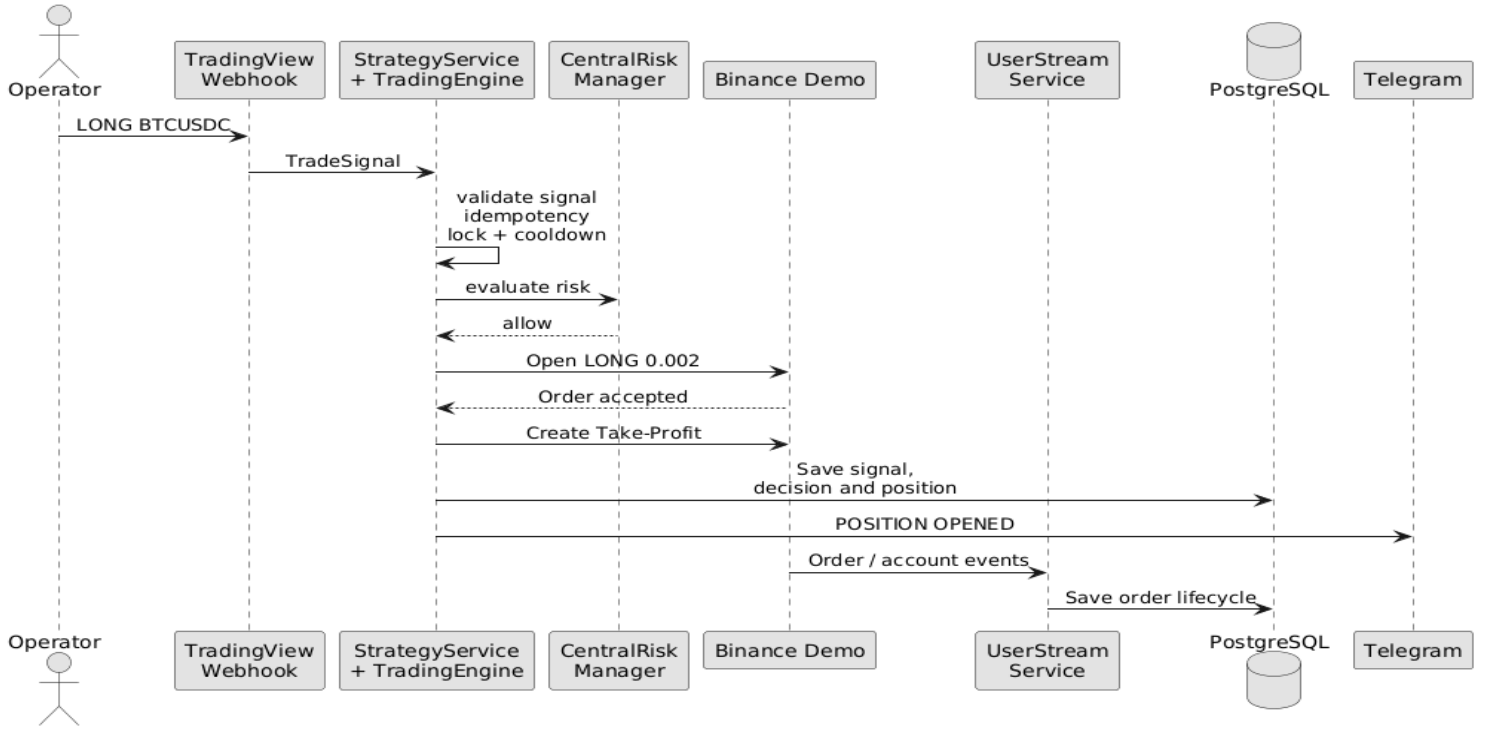


6.18. End-to-end поток на BOT8012 Demo

Runtime управление на BOT8012



BOT8012 Demo end-to-end сценарий



ГЛАВА 7. ОЦЕНКА НА СИСТЕМАТА, ОГРАНИЧЕНИЯ И НАСОКИ ЗА РАЗВИТИЕ

7.1. Оценка на постигнатата реализация

Разработената система покрива основната цел на дипломната работа: изграждане на платформа за автоматизирана търговия, наблюдение и анализ на криптоактиви, в която пазарните данни, сигналите, стратегиите, управлението на риска, изпълнението и последващият lifecycle са разделени в ясно дефинирани компоненти. Практическата стойност на решението не се състои само в способността да се отвори позиция, а в контролираната последователност от проверки и записи преди и след exchange side effect.

Една от най-силните страни на архитектурата е общият TradingEngine pipeline. TradingView сигналите и вътрешните generators използват единен TradeSignal модел, а Paper и Binance execution се включват след общите safety проверки. Това намалява дублирането и позволява една и съща стратегия да бъде наблюдавана, симулирана и изпълнявана в Demo без отделни логически пътища.

Отделянето на control plane от execution слоя също е съществено. Dashboard Web няма директен достъп до Binance credentials или Redis state. Операторските команди се записват и обработват асинхронно, а read страниците работят върху projections. Това позволява интерфейсът да предоставя богата наблюдаемост, без да се превръща в пряк транспорт към борсата.

7.2. Силни страни на архитектурата

Първата силна страна е ясното разделение между transient runtime communication и durable persistence. Redis е използван за low-latency Pub/Sub, locks, idempotency, cooldown и throttle, докато PostgreSQL пази trading history, runtime configuration, Event Store, jobs, analytics, heartbeats, alerts и audit. Това позволява всеки тип данни да бъде съхраняван според своя lifecycle.

Втората силна страна е разграничаването на strategy, risk и execution. Strategy policy отговаря дали конкретната търговска логика допуска действие. CentralRiskManager поставя системни граници върху експозицията. Executor превежда одобреното решение към Paper или Binance операция. Това разделение позволява нов бот да използва съществуващите safety механизми, без да ги имплементира повторно.

Третата силна страна е проследимостта. Signal history, strategy decisions, positions, order events, Event Store, command history, alerts и audit изграждат различни гледни точки към една и съща операция. Това улеснява troubleshooting, reconciliation и последващ анализ.

7.3. Ограничения на използвания комуникационен модел

Независимо от реализираните защитни механизми системата остава разпределена и зависи от външна борса. Redis Pub/Sub е подходящ за ниска латентност, но не предоставя durable delivery. Ако consumer е недостъпен точно в момента на публикуване, transient съобщението може да бъде пропуснато. Критичните факти поради това не трябва да съществуват само като Redis message.

Подобно ограничение съществува и при REST комуникацията с Binance. Network timeout не доказва, че заявката е отхвърлена. Поръчката може вече да е приета, а отговорът да не е достигнал до приложението. Това прави blind retry опасен. Предвидимите client order identifiers, последващ lookup и reconciliation намаляват риска, но не превръщат външната операция в локална атомарна транзакция.

7.4. Ограничения на историческия анализ

Backtesting и optimization са ценни инструменти, но резултатите им трябва да бъдат интерпретирани внимателно. Historical candles не възпроизвеждат напълно реалната ликвидност, latency, order-book microstructure и изпълнението при резки движения. Fees и slippage се моделират чрез assumptions и могат да се различават от реалните условия.

Optimization носи и риск от overfitting. Конфигурация, която е най-добра върху един исторически диапазон, може да е прекалено пригодена към конкретните движения в него. Поради това walk-forward анализът и out-of-sample проверката са по-подходящи от избиране на един-единствен trial само по най-висок score.

7.5. Fail-closed като принцип на безопасност

Когато системата няма достатъчно информация за безопасно автоматично действие, предпочитаният подход е fail-closed. Неизвестното състояние не се интерпретира като разрешение за нов entry. При критично reconciliation finding, неясен exchange outcome или липса на необходима protective state новите действия трябва да бъдат блокирани или насочени към операторска проверка.

Този подход е консервативен, но е подходящ за автоматизирана търговия. Временно пропусната възможност има ограничен ефект, докато неправомерно повторена или неконтролирана борсова операция може да създаде реална експозиция. По същата причина notification механизмите са отделени от critical execution path: липсващо Telegram съобщение не трябва да променя вече потвърден trading state.

7.6. Операторски интерфейс и човешки фактор

Dashboard намалява нуждата операторът да работи директно с PostgreSQL, Redis или exchange credentials, но човешкият фактор остава важен. Runtime commands и configuration updates могат да променят поведението на системата, затова reason, audit trail, version checking и explicit confirmation са необходими защитни механизми.

UX може да бъде допълнително подобрен. Свободните текстови филтри за bot, symbol и status са функционални, но dropdown или autocomplete би намалил вероятността за грешно въвеждане. При рисковни действия интерфейсът може да визуализира по-ясно текущата позиция, очаквания ефект от командата и exchange средата преди потвърждение.

7.7. Посоки за развитие на event delivery и persistence

Една естествена посока за развитие е transactional outbox модел за събития, които трябва да бъдат публикувани след database transaction. Това би намалило прозореца, в който локалният запис е успешен, но последващото event publication се проваля. Outbox worker може да публикува непрехвърлените събития и да ги маркира като доставени.

Event Store може да бъде използван по-широко за rebuild на projections и по-богат replay. Versioning на event contracts и deterministic projection rebuild биха улеснили възстановяването след промяна на read модела. За дългосрочно съхранение може да се добави архивиране или partitioning при значително нарастване на event volume.

7.8. Посоки за развитие на reconciliation и recovery

Reconciliation може да бъде разширен с по-богат operator workflow. Findings могат да бъдат групирани по severity, bot и symbol, а за всяко отклонение да се визуализира сравнение между local

state и exchange state. Auto-heal трябва да остане ограничен до детерминирани и доказуемо безопасни случаи.

При restart recovery може да се добави по-ясна фаза на startup reconciliation преди активиране на нови entries. Така ботът първо ще възстанови runtime контекст, ще сравни локалните позиции и orders с Binance и едва след това ще премине към Running.

7.9. Развитие на risk management

Текущият central risk слой може да бъде разширен към по-богата portfolio-level оценка. Освен абсолютен брой позиции и общ notional могат да се използват лимити по symbol, по strategy family, по посока и по групи корелирани активи. Това ще позволи рискът да се оценява не само за отделния entry, а за ефекта върху целия портфейл.

Друга посока е динамично адаптиране на лимитите спрямо drawdown, реализиран PnL и volatility regime. При влошаващ се дневен резултат максимално допустимата експозиция може да бъде намалявана, докато при нормално състояние системата да работи с базовите ограничения.

7.10. Развитие на analytics и optimization

Analytics слойът може да бъде разширен с автоматично сравнение между Backtest, Paper и Demo execution. Ако една конфигурация показва силно различни резултати между тези среди, разликата може да бъде използвана като сигнал за нереалистични assumptions, execution latency или зависимост от конкретен market regime.

Optimization може да включи повече search стратегии, ранно спиране на очевидно лоши trials и автоматично сравнение на няколко objective функции. При това е важно strategy configuration snapshot-ът да остане versioned и свързан с всеки performance run, за да може резултатът да бъде възпроизведен.

7.11. Production-ready deployment

Преминването от Demo към реална production среда изисква допълнителни operational мерки. Secrets трябва да се управляват чрез специализиран secret store, а не само чрез локален .env файл. Необходими са редовни backups, контролирано възстановяване, retention policy за logs и metrics и наблюдение на ресурсите на host машината.

За по-висока наличност могат да се разгледат отделни инстанции на Redis и PostgreSQL, backup replication и автоматизиран health monitoring. Самият trading execution трябва да остане изрично активиран, а не да се включва автоматично след deployment. Така production режимът ще бъде съзнателна operational стъпка.

ЗАКЛЮЧЕНИЕ

В дипломната работа е разработена система за автоматизирана търговия, наблюдение и анализ на крипто активи, която обединява няколко основни направления – получаване и обработка на пазарни данни, изчисляване на технически индикатори, генериране и приемане на търговски сигнали, изпълнение на стратегии, управление на риска, работа с исторически данни и операторски контрол чрез уеб интерфейс.

Основната цел на разработката е търговският процес да не бъде разглеждан като единична заявка към външна борса, а като последователност от ясно разграничени стъпки. Всеки TradeSignal преминава през проверки за валидност, runtime състояние на бота, идемпотентност, cooldown, стратегия и централизирана оценка на риска, преди да достигне до execution слоя. По този начин логиката за конкретна стратегия е отделена от общите защитни механизми на системата.

Съществена част от реализацията е разделението между различните execution среди. Paper режимът позволява проверка на поведението без външна поръчка, докато Binance Demo използва реалната интеграция с борсата, но без използване на реални средства. Това дава възможност една и съща логика да бъде проверявана постепенно и по-контролирано, преди да бъде използвана в по-рискова среда.

Интеграцията с Binance е реализирана чрез комбинация от REST и WebSocket комуникация. REST операциите се използват за създаване и управление на поръчки, а UserStream предоставя последваща информация за тяхното състояние. Така системата може да проследява не само първоначалното изпращане на поръчката, но и нейния реален lifecycle – например създаване, изпълнение, отмяна или промяна на състоянието.

Важна роля има и persistence слойът. PostgreSQL съхранява търговска история, runtime конфигурации, позиции, събития, аналитични резултати и operational информация. Redis се използва за по-бързи runtime операции и комуникация между отделните компоненти. Това разделение позволява бързите transient операции да не натоварват трайното хранилище, а важната информация да остане налична след рестарт на услугите.

Системата включва и механизми за по-добра надеждност при работа в разпределена среда. Идемпотентността ограничава риска един и същ сигнал да бъде изпълнен повече от веднъж. Operation lock предпазва от едновременни конфликтни операции, а cooldown механизмът ограничава прекалено честото отваряне на нови позиции. Reconciliation се използва за сравнение между локалното състояние и това, което реално се намира в Binance, когато е възможно да има разминаване след рестарт, прекъсване или мрежов проблем.

Разработен е и аналитичен слой, който включва работа с исторически данни, backtesting, optimization и walk-forward анализ. Това позволява различни конфигурации на стратегиите да бъдат сравнявани върху еднакви исторически периоди и резултатите да се съхраняват за последващ анализ. При BOT8012 например могат да се изследват параметри като PriceDistance, ProfitDistance, OrderSideLimit и CooldownSeconds. По този начин системата не се използва само за изпълнение на сделки, а и като среда за експериментиране и оценка на стратегии.

Отделен компонент на разработката е операторският Dashboard. Чрез него могат да се наблюдават runtime състоянието на ботовете, получените сигнали, отворените позиции, резултатите от аналитични операции, health състоянието на услугите и историята на операторските команди. Това позволява системата да бъде управлявана без директна работа с базата данни, Redis или вътрешните конфигурационни файлове.

Практическата верификация показва, че отделните компоненти могат да работят като общ end-to-end процес. При демонстрационен сценарий с BOT8012 сигнал от TradingView преминава през TradingEngine, проверките за стратегия и риск, след което достига до Binance Demo. Създава се

LONG позиция и автоматична take-profit поръчка, а резултатът се визуализира в Dashboard и се потвърждава чрез Telegram notification. По този начин може да се проследи целият процес от входния сигнал до реалното състояние в Demo средата.

Разгръщането чрез Docker и Docker Compose допълнително улеснява стартирането на системата и изолирането на отделните услуги. Използването на Synology NAS като демонстрационен host позволява приложението да работи независимо от development машината и да бъде наблюдавано в по-близка до реална среда.

Разработената система има и ограничения. Работата с външна борса означава, че винаги съществуват ситуации като network timeout, временна недостъпност или забавено събитие, при които локалното състояние не може да бъде напълно атомарно синхронизирано с Binance. Историческите симулации също не могат напълно да възпроизведат реалната ликвидност, latency и пазарна динамика. Поради това резултатите от backtesting и optimization трябва да се разглеждат като инструмент за анализ, а не като гаранция за бъдеща доходност.

В бъдеще системата може да бъде разширена с по-богати risk management механизми, по-детайлна portfolio-level оценка, по-автоматизирано reconciliation поведение, подобро управление на secrets и по-разширена observability инфраструктура. Допълнително могат да се развият аналитичните модули и да се добави по-подробно сравнение между Backtest, Paper и Demo резултатите.

Като цяло разработката показва, че изграждането на система за автоматизирана търговия изисква значително повече от реализацията на една търговска стратегия. Необходимо е съчетаване на архитектура, управление на състоянието, надеждност, persistence, външна интеграция, анализ и операторски контрол. Именно тази обща инженерна рамка е основният резултат от дипломната работа и предоставя добра основа за бъдещо развитие на системата и добавяне на нови стратегии и функционалности.

ИЗПОЛЗВАНА ЛИТЕРАТУРА И ИЗТОЧНИЦИ

1. **Binance Developers – USDⓈ-M Futures API Documentation**
<https://developers.binance.com/en/docs/products/derivatives-trading-usds-futures/>
2. **Redis – Pub/Sub Documentation.**
<https://redis.io/docs/latest/develop/pubsub/>
3. **Nick Chapsas – How to effectively use Redis Cache in .NET Core**
<https://www.youtube.com/watch?v=jwek4w6als4>
4. **Distributed Caching In ASP.NET Core With Redis | Introduction**
<https://www.youtube.com/watch?v=Tt5zIKVMMbs>
5. **Chad Thackray – AlgoTrading For Beginners – Full Course in Python**
<https://www.youtube.com/watch?v=w0wKN5L1HYI>
6. **Binance Futures Trading for Beginners 2026**
<https://www.youtube.com/watch?v=yTXSsPP163w>
7. **Pepperstone – Backtesting Algorithmic Trading Strategies**
<https://www.youtube.com/watch?v=hgW2459Qeik>
8. **Kotak Stockshaala – Why Backtesting Matters in Algo Trading**
<https://www.youtube.com/watch?v=tPu2EppPfC8>
9. **Binance Futures Testnet – practically-oriented setup video**
https://www.youtube.com/watch?v=R5EoOt_JLdo
10. **tapaScript – React Query Tutorial 2026 (TanStack Query v5)**
https://www.youtube.com/watch?v=Hu1dtgK_CkU
11. **Redis – Pub/Sub Messaging Use Case**
<https://redis.io/docs/latest/develop/use-cases/pub-sub/dotnet/>
12. **Microsoft – .NET Documentation**
<https://learn.microsoft.com/en-us/dotnet/fundamentals/>
13. **Microsoft – ASP.NET Core Documentation**
<https://learn.microsoft.com/en-us/aspnet/core/?view=aspnetcore-10.0>
14. **PostgreSQL – Official Documentation**
<https://www.postgresql.org/docs/>
15. **Docker – Docker Compose Documentation**
<https://docs.docker.com/compose/>
16. **React – Official Documentation**
<https://react.dev/learn>
17. **TanStack Query – Official Documentation**
<https://tanstack.com/query/latest/docs/framework/react/overview>
18. **TypeScript – Official Handbook**
<https://www.typescriptlang.org/docs/handbook/intro.html>
19. **xUnit.net – Documentation**
<https://xunit.net/docs/getting-started/v3/getting-started>
20. **Dapper – Official Repository and Documentation**
<https://github.com/DapperLib/Dapper>

ПРИЛОЖЕНИЕ

Пълният изходен код на разработената система „Система за автоматизирана търговия, наблюдение и анализ на криптоактиви“, включително backend услугите, операторския Dashboard, database migrations, Docker конфигурацията и тестовите проекти, е достъпен в публично GitHub хранилище на адрес: <https://github.com/yassyolo/binance-trading-system>